Multi-Agent Collaborative Question Answering Systems Based on Large Language Models: A Comprehensive Survey

ARTICLE INFO

Keywords:
Multi-agent systems
Question answering
Large language models
LLM agents
Retrieval-augmented generation
Survey

Abstract

Large language models (LLMs) have enabled question answering (QA) systems to use multiple interacting agents, but evidence about when collaboration is useful remains fragmented. This paper presents a structured evidence-mapping review of multi-agent collaborative QA based on 78 cited sources from an 85-record bibliography pool, including 13 core empirical studies published through July 2026. We first classify systems by the distribution of decision authority and control into flat, hierarchical, and hybrid architectures. We then develop a qualitative, two-phase architecture-selection approach that uses task complexity and domain span, process standardisation and dynamism, cost and resource burden, and risk tolerance and auditability as its criteria. A diagnostic comparison with ten published systems shows that the approach makes design assumptions explicit while also exposing cases in which competing requirements make a single categorical assignment inadequate.

The review further examines five enabling dimensions—agent communication, task decomposition and scheduling, knowledge management, memory management, and reasoning mechanisms. It organizes development and evaluation resources into foundation components, orchestration frameworks, application resources, and benchmarks and evaluation tools. The survey distinguishes answer-level performance from collaboration-process quality and identifies priorities for architecture-aware selection, joint coordination, provenance-aware knowledge and memory management, mechanism-level evaluation, human–agent collaboration, and continual learning. The selection approach is therefore presented as a transparent screening device, not as an externally validated performance model or a universal recommendation for multi-agent QA.

1. Introduction

Automatic question answering (QA) systems aim to enable users to pose natural-language questions and receive accurate, concise answers from a computer system. From early rule-based expert systems to modern deep-learning architectures grounded in information retrieval and machine reading comprehension, QA has undergone a remarkable transformation. In recent years, large language models (LLMs) such as GPT series (Brown et al., 2020) (OpenAI, 2023) have demonstrated extraordinary natural-language understanding and generation capabilities, ushering QA systems into a new, agent-centric era. Nevertheless, QA systems built on a single LLM agent continue to face fundamental limitations. *First*, single agents are prone to hallucination, producing factually incorrect outputs with high confidence. *Second*, finite context windows impede reliable reasoning over long documents or multi-hop inference chains. *Third*, a single model may lack sufficient domain coverage for a query that requires several areas of expertise.

At the same time, multi-agent collaborative paradigms have emerged, drawing theoretical inspiration from Minsky's "Society of Mind" (Minsky, 1988) and the formal foundations of multi-agent systems (Wooldridge & Jennings, 1995). For example, influential frameworks such as AutoGen (Wu et al., 2024) and MetaGPT (Hong et al., 2024) illustrate the emergence of explicit multi-agent coordination. Through role specialization, mutual debate (Du et al., 2024), and structured task decomposition, multi-agent systems can improve answer verification and task performance.

Although several surveys on LLM-based agents exist (Guo et al., 2024; Tran et al., 2025; L. Wang et al., 2024), they primarily address general agent architectures, communication, orchestration, and foundational capabilities, with some coverage of QA. A systematic account of collaboration topologies, enabling mechanisms, task and governance requirements, and architecture-selection trade-offs for QA remains limited. This survey addresses that gap through five related contributions:

1. We construct a structured evidence map of multi-agent collaborative QA, based on a source-checked corpus with explicit roles for core empirical studies, supporting research, background literature, and technical documentation. The review distinguishes direct QA evidence from adjacent agent applications.
2. We develop an operational architecture account that classifies systems by decision authority and control into **flat**, **hierarchical**, and **hybrid** patterns, and combines the taxonomy with a qualitative, two-phase selection approach. The approach uses a single-agent baseline, explicit decision criteria, and a diagnostic comparison with ten published systems to identify its boundary conditions.
3. We synthesize five technical dimensions that support collaborative QA: agent communication, task decomposition and scheduling, knowledge management, memory management, and reasoning mechanisms. The discussion distinguishes the functions and trade-offs of these mechanisms rather than treating them as interchangeable capabilities.
4. We organize development and evaluation resources into four layers: foundation components, orchestration frameworks, application resources, and benchmarks and evaluation tools. This map clarifies the distinction

between implementation support, application configuration, and evidence for answer-level or collaboration-process performance.
5. We synthesize open challenges and future research directions around architecture adaptation, communication and task coordination, knowledge and memory lifecycle, reasoning-process evaluation, human–agent collaboration, and continual learning.

The remainder of the paper follows this logic. Section 3 presents the architecture taxonomy and selection approach; Section 4 reviews the enabling technologies; Section 5 organizes development and evaluation resources; and Section 6 discusses open challenges and future directions.

1.1. Literature search and evidence-mapping protocol

1.1.1. Corpus construction and verification

This survey uses a *structured evidence-mapping review*. It does not seek to conduct an exhaustive PRISMA-style systematic review, as its objective is to synthesize multiple types of sources—empirical multi-agent QA studies, related agent research, foundational literature, and evolving technical documentation—with explicitly defined roles for each category of evidence.

The historical audit log records an initial candidate set of 108 unique entries from the authors' working bibliography and checks dated 2 August 2026 using Crossref, OpenAlex, arXiv, and official sources. It records four corrupted or mismatched records for which an authoritative source was not confirmed, together with duplicate, uncited, and out-of-scope records. These historical records are retained in the audit trail; the log alone is not independent evidence that every publication field was verified.

The historical working records list ReConcile (J. Chen et al., 2024), Chain-of-Agents (Y. Zhang et al., 2024), and CoCoA (Y. Jiang et al., 2026) as additions. The source audit confirms their identities and publication dates; it does not reconstruct the original update queries, retrieval results, or search dates.

Source-level checks on 8 and 10 October 2026 confirmed corrections in an 85-record bibliography pool with 78 actively cited sources; unavailable optional fields and source discrepancies remain documented in the audit trail.

1.1.2. Eligibility criteria and evidence classification

Core included studies (role **I**) had to satisfy five criteria: (1) publication from January 2019 through July 2026; (2) an LLM serving as the primary agent backbone; (3) at least two agents equipped with an explicit collaboration mechanism; (4) QA, retrieval-augmented answering, conversational QA, or question-driven domain decision support as the primary evaluated task; and (5) a described method or system accompanied by empirical evaluation. QA may be a co-primary evaluated task, but incidental QA demonstrations are insufficient. When both a preprint and a peer-reviewed version of a work existed, the peer-reviewed final publication was preferred.

Sources that informed collaboration mechanisms or applications but did not meet every core criterion were coded as supporting studies (role **S**). Foundational models, datasets, single-agent methods, and review articles were coded as background (role **B**); official protocols and software documentation were coded as technical sources (role **T**). Historically excluded sources remain coded as **X** in the audit file. The current bibliography pool contains 85 records: 13 I, 17 S, 46 B, and 9 T. The effective manuscript cites 78 of these records: 13 I, 15 S, 42 B, and 8 T; seven uncited records remain in the pool. The 13 role-I studies provide the empirical basis for the architecture taxonomy and subsequent evidence synthesis; other claims are supported by sources appropriate to their scope. Triad uses a single LLM-based agent in three roles and is coded as a supporting study (role S), because it does not meet the core criterion of at least two agents. It remains a related boundary example in Table 3, outside the 13 core studies.

Fig. 1 shows the current bibliography pool, its cited and uncited records, and the evidence-role composition of cited sources.

2. Background: automatic QA and LLM-based agents

2.1. Overview of automatic QA systems

Automatic question answering is a core branch of natural language processing (NLP) that interprets a user's information need and generates an answer. Its major paradigms, including rule-based and knowledge-base QA, information-retrieval (IR) QA, machine reading comprehension (MRC), retrieval-augmented generation (RAG), and LLM-based agentic QA, have accumulated rather than supplanted one another. They differ in their primary evidence sources, reasoning interfaces, and operational assumptions. For example, dense retrieval enables open-domain evidence access (Karpukhin et al., 2020), whereas RAG combines retrieved evidence with generative answering (Lewis et al., 2020). Multi-agent systems introduce an explicit coordination layer when a task demands distributed retrieval, reasoning, verification, or tool use. Accordingly, the historical overview in this section should be regarded as a collection of representative milestones, rather than a single linear sequence of superseded paradigms. QA has also expanded from single-turn, closed-domain scenarios to multi-turn and open-domain settings. Benchmark datasets have been instrumental in driving research progress. Representative examples include HotpotQA (Z. Yang et al., 2018) for multi-hop reasoning, Natural Questions (Kwiatkowski et al., 2019) derived from real search queries, and CoQA (Reddy et al., 2019) and QuAC (Choi et al., 2018) for conversational QA. These resources have facilitated advances in retrieval, generation, and complex reasoning. The representative milestones are illustrated in Fig. 2.

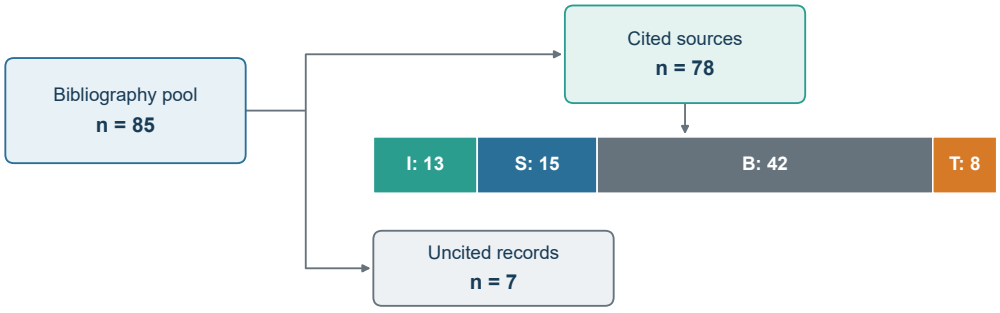


Figure 1: Current bibliography pool and evidence-role composition of cited sources. The pool contains 85 records: 78 cited sources and seven uncited records. Under the authors' evidence-role classification, cited sources comprise 13 core studies (I), 15 supporting studies (S), 42 background sources (B), and eight technical resources (T); segment widths in the horizontal bar are proportional to these counts. AI tools assisted with plotting code and layout; counts are computed from the bibliography, active citations, and evidence-role table.

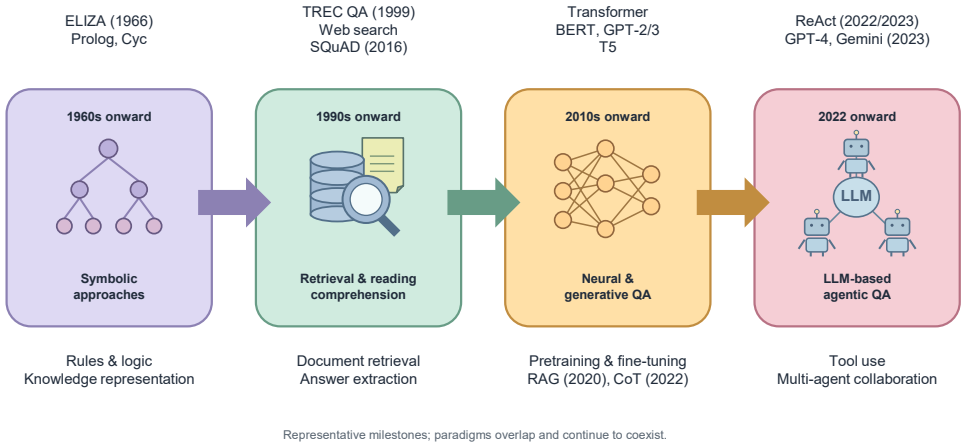


Figure 2: Evolution timeline of automatic QA systems.

2.2. LLM-based agents

2.2.1. Core architecture of LLM-based agents

An LLM-based agent takes a large language model as its cognitive core and may be equipped with perception and action capabilities. For the analytical purposes of this review, we characterize agent functionality via four capabilities: *Perception* (receiving environmental input), *Planning* (task decomposition and goal setting), *Action* (tool invocation and external interaction), and *Memory* (short-term context management and long-term knowledge retrieval). This constitutes an analytical framework, rather than implying that every agent system implements these four identical components. For instance, ReAct specifically interleaves reasoning and action (Yao, Zhao, et al., 2023). Formally, a multi-agent collaborative QA system can be defined as:

$$\mathcal{S} = \langle \mathcal{A},\ \mathcal{C},\ \mathcal{T},\ \mathcal{E} \rangle \tag{1}$$

where $\mathcal{A} = \{a_1, a_2, \ldots, a_N\}$ is the set of agents with each $a_i = \langle \pi_i, \mathcal{M}_i, \mathcal{K}_i \rangle$ denoting its planning policy, memory state, and knowledge base; $\mathcal{C}$ is the communication protocol; $\mathcal{T}$ is the task space; and $\mathcal{E}$ is the shared execution environment.

The trajectory of LLM development has been decisive in enabling this architecture. Representative milestones include the Transformer (Vaswani et al., 2017), which introduced an attention-only sequence-transduction architecture; BERT (Devlin et al., 2019), which advanced bidirectional pretraining and task-specific fine-tuning; and GPT-3 (Brown et al., 2020) and GPT-4 (OpenAI, 2023), which demonstrated powerful few-shot and zero-shot capabilities. Open-weight releases such as LLaMA (Touvron et al., 2023) and ChatGLM (Team GLM, 2024) provide additional options for local experimentation, while InstructGPT (Ouyang et al., 2022) exemplifies the use of Reinforcement Learning from Human Feedback (RLHF) to improve instruction following. Table 1 summarizes representative model capabilities relevant to agentic QA; this table serves as an illustrative overview rather than an exhaustive chronological record.

Table 1
Representative LLM capabilities relevant to agentic QA. Models were selected to illustrate capabilities used by agent systems, rather than to provide an exhaustive model history.

Model	Year	Organisation	Capability relevant to agentic QA
BERT	2018	Google	Contextual language representations supporting language understanding
GPT-3	2020	OpenAI	In-context learning for flexible task specification
InstructGPT	2022	OpenAI	Instruction following through RLHF-based alignment
LLaMA series	2023+	Meta	Open-weight model family supporting reproducible agent experimentation
GPT-4	2023	OpenAI	Strong general reasoning and multimodal interaction capabilities
ChatGLM series	2023+	THU/Zhipu	Bilingual and long-context capabilities for multilingual or document QA

2.2.2. Rationale for the transition from single-agent to multi-agent systems

Single-agent architectures remain a strong baseline for tasks that can be handled by one workflow, but their limitations become apparent when a problem requires cross-domain knowledge, long-horizon reasoning, extensive evidence, or high reliability. A single model must understand the query, retrieve and assess evidence, generate an answer, and possibly verify its own output. This concentration of functions creates three related risks: incomplete knowledge and hallucination in unfamiliar domains, dilution of relevant evidence in long contexts, and the absence of an independent perspective in a one-pass answer. Self-refinement and tool use can mitigate these risks, but they do not remove the underlying dependence on one model and one reasoning trajectory.

The motivation for multi-agent systems is therefore capability extension rather than simple replacement. Inspired by Minsky's Society of Mind (Minsky, 1988), a multi-agent architecture decomposes a complex task and assigns subtasks to agents with distinct responsibilities. Park et al.'s social simulations illustrate individual memory, reflection, and planning (Park et al., 2023). A single "generalist" workflow may combine query understanding, retrieval, generation, and refinement, whereas a multi-agent workflow can separate these functions into, for example, Planner, Retriever, Generator, and Critic roles. MetaGPT demonstrates role allocation among product managers, architects, and engineers in software development (Hong et al., 2024), while CAMEL studies sustained interaction between user and assistant roles (Li et al., 2023). These examples show how role conditioning can structure collaboration, but they do not establish a universal advantage: specialization is useful only when roles have genuinely different information, tools, or verification responsibilities.

This division of labour can extend both knowledge coverage and effective context capacity. Different agents may use complementary prompts, tools, or evidence sources, so diversity is beneficial when it contributes independent evidence rather than duplicated model calls. Likewise, segmenting a large document across agents can support broader coverage than a single context, as explored by LongAgent and Chain-of-Agents (Y. Zhang et al., 2024; Zhao et al., 2024). These gains are conditional: intermediate results must still be communicated and synthesized, and aggregation loss, communication cost, and the final synthesis context can become new bottlenecks.

Multi-agent collaboration can also improve reasoning reliability by creating separate candidate, review, and verification paths. Debate, cross-checking, and reconciliation may expose errors that self-refinement within one workflow overlooks (J. Chen et al., 2024; Du et al., 2024; Xiong et al., 2023). However, additional agents do not guarantee correctness: weak aggregation, poor interaction design, or correlated errors can preserve or amplify an incorrect answer. Coordination quality is thus a central design variable, not an incidental implementation detail.

3. Architectures of multi-agent collaborative QA systems

3.1. Taxonomy of multi-agent collaborative QA architectures

This review classifies a collaborative QA system primarily by the distribution of *decision authority and control*. The classification is based on the workflow that routes subtasks, determines when an answer is accepted, and controls final synthesis. A **flat** architecture has no persistent agent with top-down authority over the other participants. A **hierarchical** architecture assigns planning, routing, approval, or synthesis authority to one or more managers, or imposes an ordered, asymmetric control flow across stages. A **hybrid** architecture changes this authority structure across task stages or decision states. Other design considerations such as communication topology (for example, broadcast, peer exchange, or a graph), role specialization, human-in-the-loop (HITL) checkpoints, and runtime routing are orthogonal attributes. For example, a group-chat framework is not automatically flat, and a system with specialized roles is not automatically hierarchical. The three classes are alternative control patterns rather than stages of a historical progression. Figure 3 provides an illustrative schematic.

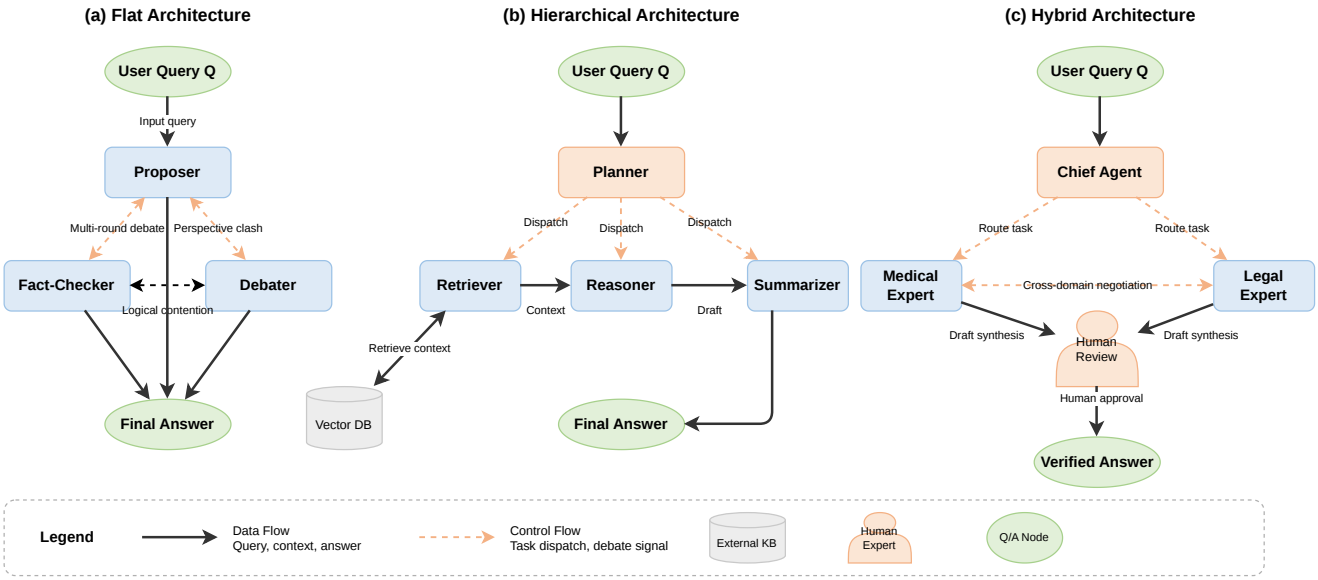


Figure 3: Illustrative network topologies for multi-agent collaborative QA. The operational taxonomy is defined by decision authority and control, as specified in the text and Table 2.

3.1.1. Flat architectures

In a flat collaborative architecture, participating agents exchange information, critique candidate solutions, or engage in negotiations without a persistent top-down controller. The relevant characteristic lies in distributed decision-making authority, rather than unrestricted dialogue. A flat workflow may still enforce turn limits, adopt voting rules, or include a designated final aggregation step.

Multi-agent debate. In the multi-agent debate paradigm, several LLM agents generate candidate answers and critique one another across one or more rounds. Representative studies illustrate both the potential and the limits of this design. For example, Du et al. (2024) report improved factuality and reasoning performance on the tasks they evaluate, whereas divergent prompting can mitigate convergence toward similar reasoning paths (Liang et al., 2024). ReConcile provides another example by integrating heterogeneous models with confidence-weighted consensus (J. Chen et al., 2024); inter-model inconsistency and capability imbalance nevertheless remain important considerations (Xiong et al., 2023). Accordingly, adding more agents or debate rounds does not reliably boost QA performance; instead, the interaction rules, diversity of evidence, and aggregation procedure determine whether disagreement yields correction, inconsistency, or unnecessary computational cost.

Role-conditioned collaboration. Role-conditioned collaboration assigns different prompts or task views to participating agents. For example, CAMEL (Li et al., 2023) employs an "AI User" and an "AI Assistant" paired with inception prompting to maintain role-conditioned interaction. This role pair has asymmetric instructions; role play alone does not establish a flat decision-authority structure. This constitutes an adjacent multi-agent framework and does not serve as direct evidence that role playing improves QA accuracy.

Group-chat collaboration. Group-chat collaboration adopts multi-participant conversations as its interaction interface. For example, AutoGen (Wu et al., 2024) supports developer-defined speaker selection rules and conversation patterns instead of enforcing a fully flat, free-form group chat paradigm. Accordingly, its resulting topology is determined by application configurations.

Flat architectures can be useful when independent viewpoints, mutual critique, or open-ended exploration are required. Their distributed structure can reduce reliance on a single reasoning path, yet it also incurs coordination costs. In fully connected settings, the number of potential communication links grows quadratically with the number of agents. Additional rounds may introduce redundant context, delayed convergence, correlated errors, or pseudo-consensus. By contrast, when tasks demand stable dependencies, predictable execution, or stringent cost control, explicit authority and routing mechanisms may be more appropriate.

3.1.2. Hierarchical architectures

Hierarchical architectures delegate the authority for planning, routing, approval, or synthesis to one or more agents. Lower-level agents execute bounded subtasks or provide evidence to a manager. The workflow may take the form of a tree, a pipeline, or a controller–worker loop; its defining characteristic is asymmetric decision-making authority.

Manager–executor pattern. The manager–executor pattern separates global planning from local execution. For example, MetaGPT (Hong et al., 2024) adopts standard operating procedures and defines roles including product manager, architect, and engineer. ChatDev (Qian et al., 2024)

provides another example by simulating a software development organisation via ChatChain. Both fall under related software engineering systems; they demonstrate structured delegation yet do not provide direct evidence regarding QA performance.

Plan–retrieve–generate pipeline. Knowledge-intensive QA can assign planning, retrieval, generation, and verification to a staged workflow. Representative implementations include LONGAGENT (Zhao et al., 2024), which coordinates a leader with document-reading members; Chain-of-Agents (Y. Zhang et al., 2024), which propagates information across context segments before synthesis; and CoCoA-zero (Y. Jiang et al., 2026), which coordinates parametric and retrieved knowledge using three agents at inference time. CoCoA subsequently distils that collaboration into a single model. These role-I systems differ in their communication and aggregation mechanisms, but each places subtask sequencing or synthesis under explicit control. By contrast, HuggingGPT (Shen et al., 2023) uses a controller to invoke specialised models and tools.

Hierarchical architectures can improve controllability by separating global planning from local execution. They are suitable for QA workflows with stable procedures, explicit dependencies, or quality-control checkpoints. However, this centralisation may introduce a planner bottleneck: an erroneous high-level plan can propagate errors, and rigid pipelines may fail to handle unexpected evidence or conflicting expert opinions. Therefore, the associated benefits depend on the quality of the controller and the availability of corrective pathways.

3.1.3. Hybrid architectures

Hybrid architectures alter the control configuration across task stages or decision states. A system may adopt peer deliberation to explore candidate solutions, hierarchical delegation to execute a selected plan, and conditional human oversight in scenarios where uncertainty or risk exceeds a predefined threshold. Simply combining individual components is insufficient; the coordination paradigm itself must be adaptive.

Graph-structured agent networks. Graph-structured networks represent agents and transitions as an executable graph. For example, LangGraph (The LangChain Team, 2024a) serves as an official technical framework for defining state transitions, loops, and conditional branches; it provides technical evidence (role T), rather than constituting a QA evaluation. GPTSwarm (Zhuge et al., 2024) provides another example through an empirical study of agent methodologies that optimises prompts and inter-agent graph connections. A graph representation, edge optimisation, or conditional routing alone does not establish hybrid decision authority; the implementation must actually change who controls decisions.

Adaptive collaboration-mode switching. Adaptive systems adjust team composition or control patterns as tasks evolve. For example, AgentVerse (W. Chen et al., 2024) describes an iterative pipeline consisting of expert recruitment, collaborative decision-making, action execution, and result evaluation. This mechanism cannot be simplified to a "simple-flat/complex- hierarchical" rule. MDAgents (Y. Kim et al., 2024) routes medical queries to solo, multidisciplinary-team, or integrated-care-team pathways according to complexity under a moderator. This supports adaptive collaboration, but a persistent moderator means that mode switching alone is insufficient to prove hybrid authority. Such adaptive mechanisms enhance routing flexibility, yet introduce extra requirements for state management and performance evaluation.

Human-in-the-loop design pattern. For example, LangGraph supports pausing a workflow at defined breakpoints for human review (The LangChain Team, 2024b), which may be useful for high-risk decisions.

In a collaborative QA workflow, human participation can be placed at several points: before task decomposition, after retrieval, when agents disagree, or before an answer is released or an external action is executed. These points serve different purposes. Early review can correct the interpretation of the question and restrict the permitted tools; intermediate review can assess the quality and provenance of retrieved evidence; and final review can provide an approval boundary for consequential recommendations. The appropriate point is therefore determined by the potential impact of an error and by the extent to which the relevant state remains understandable and editable.

The reviewer should also have an explicitly defined level of authority. A human may merely approve a proposed answer, select among agent outputs, edit evidence or intermediate conclusions, or redirect the workflow by requesting another retrieval or specialist analysis. These forms of participation are not equivalent: approval without access to the supporting evidence provides limited control, whereas unrestricted editing can make the origin of the final answer difficult to reconstruct. A useful HITL design should preserve the question, retrieved sources, agent contributions, reviewer intervention, and resulting state transition in an auditable record.

HITL checkpoints can improve accountability and provide a recovery path when agents encounter uncertainty, conflicting evidence, or unsafe requests. They also introduce review latency, operator workload, and the possibility of inconsistent decisions across reviewers. For this reason, a fixed human breakpoint is not automatically preferable to automated execution. Risk- adaptive policies that trigger review using uncertainty, disagreement, evidence quality, or action impact are a plausible design, but their effects on error interception, workload, and answer quality require empirical evaluation. The current literature therefore supports HITL as a control pattern and design objective, rather than as established evidence of safer QA performance.

Hybrid architectures are useful when task requirements are heterogeneous or when the workflow must combine

Table 2
Comparison of multi-agent collaborative QA architecture classes and related mechanisms.

Architecture	Sub-paradigm	Representative systems	Coordination and control	Suitable contexts	Limitations
Flat	Multi-agent debate	Du et al. (2024)	Peer critique and aggregation can add independent answer paths	Tasks needing competing factual or reasoning views	Cost and aggregation sensitivity
Hierarchical[a]	Role-conditioned collaboration	CAMEL (Li et al., 2023)	Personas structure interaction but do not by themselves establish QA gains	Exploratory collaboration and prompt-role studies	Fixed roles or correlated perspectives
—	Group-chat collaboration	AutoGen (Wu et al., 2024)	Developer-defined speaker selection and dialogue patterns	Applications requiring configurable conversations	Topology and QA performance depend on configuration
Hierarchical	Manager–executor pattern	MetaGPT (Hong et al., 2024)	Manager assigns staged work to specialised executors	Structured engineering workflows	Planner bottleneck and error propagation
	Staged evidence-processing pipeline	LONGAGENT; Chain-of-Agents; CoCoA-zero (Y. Jiang et al., 2026; Y. Zhang et al., 2024; Zhao et al., 2024)	Explicit sequencing and final synthesis over distributed evidence	Long-document or knowledge-intensive QA	Information loss and coordination cost at synthesis
—	Graph-structured agent networks	GPTSwarm; LangGraph (The LangChain Team, 2024a; Zhuge et al., 2024)	GPTSwarm DAG connection optimisation; LangGraph transitions permit loops, branches, and routing policies	Workflows with conditional state transitions	Different evidence roles; difficult debugging and tracing
Hierarchical[b]	Adaptive collaboration-mode switching	MDAgents (Y. Kim et al., 2024)	Changes collaboration in response to case characteristics	Tasks with heterogeneous requirements	Routing policy must be evaluated, not assumed beneficial
—	Human-in-the-loop design pattern	LangGraph breakpoints (The LangChain Team, 2024b)	Human review at explicitly defined decision points	High-risk workflows requiring accountability	Direct comparative QA evidence remains limited

Note: Architecture labels concern decision authority in a specified implementation. [a]CAMEL: persistent AI User instruction and termination control in the reported AI User–AI Assistant workflow. [b]MDAgents: moderator-led routing among solo, multidisciplinary-team (MDT), and integrated-care-team (ICT) pathways. — denotes a mechanism or framework without a configuration-independent architecture label, not a fourth class. Flat, Hierarchical, and Hybrid retain the definitions in the text; graph structure, team switching, and human review alone do not establish Hybrid. A hybrid architecture requires a demonstrated change in decision authority. A fixed human approver is hierarchical.

deliberation, controlled execution, and conditional review, provided that decision authority changes across stages or decision states. Their trade-off lies in engineering complexity: dynamic routing, state transitions, and cross-mode information transfer are more difficult to configure, debug, evaluate, and reproduce. A hybrid design is justified when this additional adaptability addresses documented uncertainty or risk, rather than serving as a default replacement for fixed control patterns.

3.1.4. Comparative discussion of architecture types

Table 2 compares flat, hierarchical, and hybrid architectures and their related mechanisms. Architecture labels, suitable contexts, and limitations are the authors' synthesis; related mechanisms do not define additional architecture classes.

These architecture types differ in where decision authority is placed and when it changes. Communication topology remains a useful secondary descriptor, but it does not supersede the operational rules above.

3.2. Architecture selection approach

The taxonomy describes alternative control patterns; it does not imply that multi-agent collaboration is preferable for every QA task. Architecture selection must also account for the task, process, resource, and governance conditions under which collaboration is justified. The approach below treats a single-agent workflow as a valid baseline and produces a reasoned shortlist of candidate architectures rather than a universal assignment.

3.2.1. Scope and selection criteria

Four criteria connect task requirements to the control patterns in the taxonomy. They support transparent comparison across studies, but do not define universal numerical thresholds or claim that one architecture dominates the others.

1. Task complexity and domain span. The analysis starts from the reasoning structure, evidence requirements, output criteria, and domain span. The criterion captures the number and interdependence of reasoning steps, the breadth of required expertise, and the need for independent evidence sources. Single-agent execution is often adequate for short, low-uncertainty queries; decomposition and role specialization become more plausible when a task combines separable subtasks or domains. Complexity alone, however, does not establish that additional agents will improve the answer: each role must contribute distinct information or checks.

2. Process standardization and dynamism. Stable stages and explicit dependencies favor hierarchical or pipeline control, whereas open-ended tasks may benefit from peer generation and critique. When the appropriate mode changes

across queries or stages, hybrid routing is a candidate design. These are tendencies rather than deterministic mappings: hierarchical workflows can contain local debate, and flat workflows can enforce turn limits and aggregation rules.

3. Cost and resource burden. Multi-agent execution adds communication, aggregation, state-management, and inference costs. A collaborative design is justified only when expected gains in coverage, reliability, or adaptability offset those costs, as a criterion of this review. Reported multi-agent failure modes motivate this assessment (Cemri et al., 2025). Assessment should include latency, token use, tool calls, monitoring, and recovery overhead.

4. Risk tolerance and auditability. This criterion determines how much decision authority can be delegated and what evidence must be retained. Healthcare, legal, financial, and other consequential applications require provenance, review, contestability, and explicit responsibility. HITL checkpoints are possible controls, but their placement and scope depend on the decision and applicable governance requirements and must be evaluated rather than assumed.

3.2.2. Two-phase architecture-selection procedure

The four criteria are combined in the two-phase flow shown in Fig. 4. The flow is a screening heuristic: it narrows the plausible design space and makes trade-offs explicit, but does not replace empirical comparison of candidate implementations.

The procedure is developed as a design artifact following the Design Science Research Methodology (DSRM) (Peffers et al., 2007). Problem identification and objective definition motivate a transparent selection procedure; design and development translate the architecture and technology review into the four criteria; demonstration uses contrasting QA scenarios; and verification compares the framework's interpretations with published systems. The result is a structured design proposal whose assumptions remain open to empirical testing.

Phase 1 (pre-screening). Establish whether one workflow can meet the required quality and cost. If so, retain a single-agent baseline. If the task requires separable expertise, long-horizon reasoning, or independent verification, retain multi-agent designs for the second phase. Use risk and auditability to constrain the permissible degree of autonomy.

Phase 2 (topology selection). Compare process structure and information dependencies. Stable stages and explicit hand-offs support hierarchical control; peer generation and critique support flat collaboration when independent perspectives are valuable; hybrid control is a candidate when changing requirements necessitate a change in decision authority. Conditional review under a fixed approver remains hierarchical. Compare the selected design with a single-agent baseline using task quality, latency, cost, provenance, and failure-recovery measures.

3.2.3. Verification and illustrative applications

The approach is verified for interpretive consistency by comparing its inferred architectures with ten published systems. This purposive sample was selected because the systems report agent roles and control flow with enough detail for coding and cover different domains, process structures, and risk levels. The comparison does not evaluate system performance, establish causal benefit, or claim external validity. It contains both core QA studies and supporting workflow examples and is not independent validation. The proposed labels and criterion profiles are retained, while reported workflows are coded by decision authority. Flat control has no persistent top-down controller; hierarchical control has stable routing, approval, or final-decision authority; hybrid control requires a demonstrated change in that authority. Iteration, voting, multiple roles, and graph structure alone are insufficient.

Three mutually exclusive ratings are applied in order. A case is broadly aligned when the proposed and source-coded overall categories agree and the reported workflow does not contradict their defining control features. When the categories differ, partial alignment requires a substantive local mechanism compatible with the proposed category and an explicit overall control conflict; otherwise the case is divergent. For a flat recommendation, partial alignment requires all three conditions: (F1) at least two expert roles contribute their own analyses; (F2) they review the same report under the same acceptance or rejection rule, without an exclusive expert reviewer; and (F3) objections feed report revision and renewed review. An iteration limit is permitted, so unanimity is not guaranteed. For a hierarchical recommendation, local compatibility requires fixed delegation, routing, approval, or synthesis authority in a substantive sub-process. For a hybrid recommendation, it requires an actual change in decision-authority configuration within a reported sub-process. Insufficient method evidence would leave a case unscored and require reporting the scored denominator.

Six cases are broadly aligned, two are partially aligned, and two are divergent under the coding used here. MedAgents and LLM-MedQA retain hierarchical overall control while supporting equal-participation expert review locally. These local mechanisms explain partial alignment with a flat recommendation; they do not establish a flat or hybrid overall architecture. LegalGraphRAG and EduPlanner remain divergent. Du et al.'s peer debate is flat at the macro level but uses iterative convergence; EduPlanner and Financial Reflection QA combine convergent objectives with evaluator or critic feedback. These boundary cases show that competing requirements can make a single categorical label insufficient and that a multi-label description of control flow may sometimes be preferable, provided that overall authority and local mechanisms are distinguished. Financial Reflection QA remains hierarchical despite critic feedback. Nine reported workflows are coded hierarchical and one flat; this sample therefore does not validate the hybrid category. All ten cases have sufficient method evidence for this limited comparison. The comparison is retrospective and was not blind or preregistered; these ratings are descriptive, not prediction accuracy or evidence of an effective architecture-selection policy.

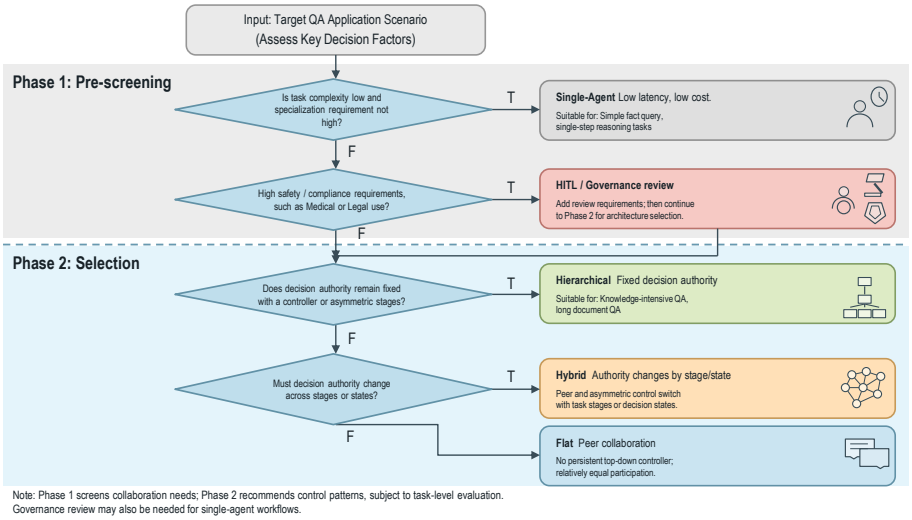


Figure 4: Two-phase decision flowchart for multi-agent QA architecture selection across application scenarios.

Table 3
Diagnostic comparison of the architecture-selection approach with ten published systems.

System	Application and criterion profile	Inferred versus source-based architecture coding	Match
MedAgents (Tang et al., 2024)	Multi-specialty medical QA; high complexity and moderate risk	Flat versus hierarchical staged synthesis	△
LLM-MedQA (H. Yang et al., 2025)	Standardised medical MCQ; convergent objective with diverse views	Flat versus hierarchical voting pipeline	△
LegalGraphRAG (Z. Chen et al., 2026)	Legal verdict prediction; high risk and structured retrieval	Hybrid versus hierarchical graph audit chain	×
Triad (Zong et al., 2024)	Knowledge-base QA; standardised Text-to-SPARQL workflow	Hierarchical versus hierarchical pipeline	✓
LONGAGENT (Zhao et al., 2024)	Ultra-long multi-hop QA; divide-and-conquer process	Hierarchical versus hierarchical leader–member workflow	✓
ChatDev (Qian et al., 2024)	Software engineering; highly standardised stages	Hierarchical versus hierarchical ChatChain	✓
AgentCoder (Huang et al., 2023)	Code generation; explicit three-role dependencies	Hierarchical versus hierarchical feedback loop	✓
Multiagent debate (Du) (Du et al., 2024)	Factual QA and reasoning; peer exchange	Flat versus flat iterative debate	✓
EduPlanner (X. Zhang et al., 2025)	Educational planning; open and convergent requirements	Flat versus hierarchical evaluator–optimizer	×
Financial Reflection QA (Fatemi & Hu, 2024)	Numerical QA; convergence with critic feedback	Hierarchical versus hierarchical critic feedback	✓

Note: ✓ = broadly aligned; △ = partially aligned; × = divergent, under the ordered rules in the text. Proposed labels are retained from the preceding draft; source-based labels and ratings are the authors' retrospective coding. Partial alignment denotes local organisational compatibility despite an overall control conflict. MedAgents was checked against its ACL 2024 method and Algorithm 1; LLM-MedQA against the IJCNN 2025 paper, Sections III-A–F and Algorithm 1 (physical pages 3–6). Neither labels nor ratings establish prediction accuracy, selection effectiveness, or safety. Triad is a single-agent, three-role supporting study (S), outside the 13 core studies. ChatDev, AgentCoder, and EduPlanner are also supporting workflow examples. Criterion profiles are illustrative author judgments.

The criteria can also be applied analytically to contrasting settings. In high-risk, cross-domain consultation, parallel specialist analysis may improve coverage, but final decisions require an explicit review boundary. A fixed review or human-approval boundary remains hierarchical; hybrid control is a plausible candidate only when decision authority changes. Standardised, evidence-grounded QA generally supports a hierarchical RAG baseline with source and citation checks. Open-ended educational or legal tasks may benefit from bounded peer critique, but legal use additionally requires auditability and human accountability. These examples illustrate how task and governance requirements constrain the design space; they do not imply one universally correct topology.

3.2.4. Summary and limitations

The approach makes architecture selection explicit through four criteria, a two-phase flowchart, illustrative cases, and a diagnostic comparison with ten published systems. Its limitations are substantial: competing requirements can make a single topology label inadequate; the framework and model ecosystem change rapidly; the purposive sample cannot establish external validity; and the criteria remain qualitative rather than calibrated on a large labelled system corpus. Independent coders, larger preregistered samples, quantitative cost and quality measures, and prospective comparisons with alternative selection procedures are needed.

4. Key technologies in multi-agent collaborative QA

The efficient operation of a multi-agent collaborative QA system depends not only on a well-designed organizational architecture but also on the support of core underlying technologies. This section investigates five technological dimensions: communication mechanisms, task decomposition, memory management, knowledge management, and reasoning mechanisms.

4.1. Agent communication mechanisms

Agent communication mechanisms determine how information is exchanged among agents during collaborative QA. They support the transfer of queries, intermediate findings, evidence, and feedback, and thereby shape coordination, traceability, and communication cost. Communication may occur through direct messages or through changes to a shared environment; interoperability protocols provide a related interface for connecting agents with tools and data sources. The following subsections distinguish these interaction mechanisms from the protocols that support cross-system communication.

4.1.1. Explicit communication

Explicit communication refers to the direct transfer of information between agents via predefined formats. The most common form is message passing based on structured languages (JSON, XML) or dedicated prompt templates. For example, AutoGen (Wu et al., 2024) provides a representative implementation: customisable conversable agents send and receive natural-language or code messages following conversation patterns defined by developers. ChatDev (Qian et al., 2024) provides another example by organising role-specialised agents through a multi-turn dialogue chain covering design, coding, and testing stages.

4.1.2. Implicit communication

Implicit communication does not require direct agent-to-agent messaging; instead, collaboration is mediated via modifications to a shared operating environment. For instance, in a code-generation QA task, one agent alters the state of a code repository or database, while another agent obtains feedback by observing compilation errors or test execution results.

This paradigm can reduce redundant message transfers, though its impact on communication overhead and fault tolerance is contingent on the implementation and task environment.

4.1.3. Standardised communication protocols

With the rapid development of multi-agent ecosystems, interoperability barriers have arisen between agents developed by different vendors. To address this issue, the industry has initiated efforts to standardize communication protocols. For example, Anthropic introduced the Model Context Protocol (MCP) (Anthropic, 2024) to standardise connections to external tools and data sources. Google's Agent2Agent (A2A) protocol (Surapaneni et al., 2025) supports communication and task coordination between agents across frameworks and vendors. These interfaces can support heterogeneous multi-agent QA systems.

Explicit messaging makes the content and provenance of an agent's contribution inspectable, and is useful when agents must exchange evidence, explain decisions, or coordinate across distinct roles. Its main cost is message volume: repeated dialogue consumes context and adds latency, particularly as the team grows. Implicit communication reduces direct message passing by using shared environmental state, but it depends on agents being able to observe and interpret that state consistently; otherwise, changes may be ambiguous or difficult to audit. Standardized protocols address a different problem. MCP and A2A aim to improve interoperability between tools, data sources, and agents, but protocol compatibility alone does not guarantee useful coordination or reliable QA. Thus, explicit and implicit communication are interaction mechanisms, whereas protocols provide an interface layer that may support either. Selection should reflect the need for traceability, the availability of shared state, and the expected interaction cost (Anthropic, 2024; Qian et al., 2024; Surapaneni et al., 2025; Wu et al., 2024).

4.2. Task decomposition and scheduling

Task decomposition and scheduling determine how a complex QA query is divided into manageable work and how that work is assigned and coordinated. Their purpose is to make interdependent reasoning steps tractable, enable

specialisation or parallel execution where appropriate, and ensure that sub-results can be combined into a coherent answer. The discussion proceeds from decomposing a query, to assigning subtasks to suitable agents, and then to optimising or adapting their execution.

4.2.1. Hierarchical task decomposition

For complex QA tasks, LLMs often cannot answer in a single step. Hierarchical task decomposition reduces difficulty by breaking complex queries into sub-tasks that can be processed in parallel or sequentially. Representative methods include Least-to-Most Prompting (D. Zhou et al., 2023) and Decomposed Prompting (Khot et al., 2023), which allow a model to solve simpler sub-problems first and then exploit their answers to resolve the ultimate target. Plan-and-Solve (L. Wang et al., 2023) provides another example by explicitly separating planning from solving and reducing missing-step errors on the evaluated reasoning tasks. Formally, given a complex query Q, a decomposition function $\mathcal{D}$ maps it to an ordered sequence of sub-tasks:

$$\begin{aligned} \mathcal{D}(Q) &\rightarrow \{q_1, q_2, \ldots, q_m\}, \\ \text{Answer}(Q) &= \mathcal{F}\big(\text{Sol}(q_1), \ldots, \text{Sol}(q_m)\big) \end{aligned} \tag{2}$$

where $\text{Sol}(q_i)$ is a sub-agent's solution to sub-task q_i and $\mathcal{F}$ is an aggregation function (e.g., voting, concatenation, or re-generation).

4.2.2. Role-based task assignment

After decomposition, the system must assign sub-tasks to the most suitable agents. For example, MetaGPT (Hong et al., 2024) organises agents through predefined software-engineering roles, while AgentVerse (W. Chen et al., 2024) recruits experts by generating task-specific expertise descriptions and adjusts team composition using evaluation feedback. These mechanisms guide which agents contribute to a task; their scheduling efficiency requires separate evaluation.

4.2.3. Pipeline optimisation and adaptive orchestration

Static assignment can be inefficient when task difficulty and agent capabilities vary across queries. For example, DSPy (Khattab et al., 2024) addresses a related but narrower problem: it represents language-model pipelines as declarative modules and compiles their prompts and demonstrations against a task metric. It should therefore be regarded as a pipeline-optimization toolkit rather than evidence supporting runtime multi-agent load balancing. Dynamic agent recruitment, routing, and early stopping remain distinct orchestration decisions that require task-specific evaluation.

Decomposition, assignment, and orchestration solve related but distinct problems. Decomposition determines which sub-questions must be answered and how they depend on one another; assignment matches those sub-questions to agents; orchestration determines their execution order and whether the workflow should adapt at runtime. Least-to-Most, Decomposed Prompting, and Plan-and-Solve primarily inform the first decision, whereas role profiles in MetaGPT and AgentVerse address the second (W. Chen et al., 2024; Hong et al., 2024; Khot et al., 2023; L. Wang et al., 2023; D. Zhou et al., 2023). A fixed sequence is adequate when dependencies and agent capabilities are known in advance; dynamic routing may be more suitable when query difficulty or available expertise varies, but it adds scheduling overhead and failure points. In either case, decomposition errors can propagate: omitted sub-questions leave gaps in the final answer, while unnecessary decomposition creates redundant calls and aggregation work. Pipeline-optimization tools such as DSPy can improve task-metric performance of language-model modules, but should not be treated as evidence that runtime recruitment or load balancing is effective (Khattab et al., 2024). Evaluation should distinguish decomposition quality, assignment fit, and orchestration efficiency rather than attributing an end-to-end gain to a single mechanism.

4.3. Knowledge management

Knowledge management concerns how external, retrieved, and agent-generated information is represented, validated, and made available to the team. It provides a shared evidence basis for collaborative QA and aims to reduce duplicated information seeking without allowing unverified content to spread. It is therefore a system-level layer built on, but not equivalent to, individual agent memory.

In complex collaboration, knowledge need not be siloed within individual agents. The system can provide a shared knowledge graph when collaborative nodes need access to critical facts. By constructing a shared graph-vector database, newly spawned sub-agents can reuse the team's cumulative knowledge, potentially reducing redundant retrieval and cognitive divergence; these effects and synchronisation latency require evaluation.

Knowledge management operates above individual memory. A shared graph or vector store can provide a common evidence base and reduce duplicated retrieval, but unrestricted sharing can also spread stale, irrelevant, or erroneous content across the team. Practical designs should attach provenance, timestamps, confidence, and validation status to shared facts, and should allow agents to revise or withdraw them. The key design questions are which knowledge is shared, which agents may access it, how conflicts are resolved, and how retrieved or generated content is verified before it becomes reusable team knowledge.

4.4. Memory management

Memory management concerns the information state retained by an agent or workflow over time. It preserves relevant context across interaction turns and supports continuity when a task spans multiple reasoning steps. It is distinct from knowledge management: memory concerns retention and recall of state, whereas knowledge management concerns the organisation, validation, and distribution of reusable information.

4.4.1. Short-term memory and working memory

Short-term memory preserves dialogue history and local context for the current QA session. For example, MemGPT (Packer et al., 2023) draws on operating-system memory

hierarchies to manage a limited main context and external memory. Function calls control information transfer between them, while a FIFO message queue and recursive summaries manage dialogue context under the context-window limit.

4.4.2. Long-term memory and vector retrieval (RAG)

Long-term memory can rely on vector databases to store historical experiences and external domain knowledge (Z. Zhang et al., 2025). A representative access mechanism is Retrieval-Augmented Generation (RAG) (Gao et al., 2023; Lewis et al., 2020), through which multi-agent QA systems can access long-term memory. In an illustrative implementation, a retrieval agent can select the Top-k relevant documents from knowledge base $\mathcal{D}$ using cosine similarity:

$$\mathrm{Rel}(q, d_i) = \frac{\boldsymbol{\phi}(q)^\top \boldsymbol{\phi}(d_i)}{\|\boldsymbol{\phi}(q)\| \, \|\boldsymbol{\phi}(d_i)\|}, \qquad \hat{\mathcal{D}} = \underset{d_i \in \mathcal{D}}{\text{Top-k}} \, \mathrm{Rel}(q, d_i) \tag{3}$$

where $\boldsymbol{\phi}(\cdot)$ is the dense encoder embedding function and $\hat{\mathcal{D}}$ is the set of retrieved relevant documents. Retrieval has evolved from passive retrieval to active variants. Self-RAG (Asai et al., 2024) uses reflection tokens to decide when to retrieve and to critique generated passages, while FLARE (Z. Jiang et al., 2023) triggers retrieval when predicted upcoming text contains low-confidence tokens and then regenerates that text. These controls can inform retrieval components in a multi-agent workflow.

Short-term and long-term memory address different temporal scopes. Working memory supports continuity within the current interaction and must prioritise information under a limited context budget; long-term memory extends access beyond that context, but depends on retrieval relevance and the quality and currency of stored experiences or documents. MemGPT illustrates session-context management, whereas RAG and its retrieval variants provide access to information outside the active context (Asai et al., 2024; Gao et al., 2023; Z. Jiang et al., 2023; Lewis et al., 2020; Packer et al., 2023). A system should therefore specify what is retained, for how long, and under what conditions memories are updated or discarded.

4.5. Reasoning mechanisms

Reasoning mechanisms aim to improve answer quality by broadening the available reasoning paths, distributing evidence processing, or revisiting provisional conclusions to detect errors. In collaborative QA, these methods can complement one another, but they target different constraints and may add inference cost. The following categories cover collaborative reasoning and answer aggregation, reasoning distributed across agents, and reflection or critique of generated answers.

4.5.1. Collaborative chain-of-thought

Chain-of-Thought (CoT) prompting (Wei et al., 2022) has greatly unlocked the reasoning potential of LLMs. Self-Consistency (X. Wang et al., 2023) provides a representative single-model method: it samples multiple reasoning paths and selects an answer by marginalising over those paths. Multi-agent systems adapt this diversity principle by assigning independent reasoners and then using debate or confidence-aware aggregation (J. Chen et al., 2024; Du et al., 2024).

In the single-model baseline, K reasoning paths $\{(r_k, a_k)\}_{k=1}^{K}$ are sampled for the same question, and majority voting selects the final answer:

$$\hat{a} = \arg\max_{a \in \mathcal{Y}} \sum_{k=1}^{K} \mathbf{1}\big[f_\theta(x, r_k) = a\big] \tag{4}$$

where $f_\theta(x, r_k)$ is the answer produced by the k-th reasoning chain and $\mathbf{1}[\cdot]$ is the indicator function. The equation describes the single-model Self-Consistency baseline; exchange between agents is an additional collaboration design, not a property of the original method.

4.5.2. Chain-of-agents reasoning

For ultra-long-text QA, the attention mechanism of conventional Transformers faces severe computational bottlenecks. Chain-of-Agents (Y. Zhang et al., 2024) distributes evidence processing across worker agents, while Tree of Thoughts (Yao, Yu, et al., 2023) and Graph of Thoughts (Besta et al., 2024) organise intermediate reasoning states as search structures. In Chain-of-Agents, worker agents process successive text chunks and pass forward condensed evidence before a manager produces the answer. LONGAGENT (Zhao et al., 2024) uses a different leader–member design in which a leader coordinates member agents over long-document segments. Tree- and graph-structured reasoning provide related search topologies, but are not necessarily multi-agent systems in their original formulations.

4.5.3. Self-reflection and mutual-critique mechanisms

To suppress hallucination, agents may use iterative self-examination. For example, Reflexion (Shinn et al., 2023) stores linguistic feedback from prior trials in episodic memory, whereas Self-Refine (Madaan et al., 2023) provides another example by iterating between generation, feedback, and revision without additional model training. Chain-of-Verification (Dhuliawala et al., 2024) drafts an answer, plans verification questions, answers them independently, and then revises the original answer. This procedure targets factual errors; dedicated critic agents are a possible multi-agent extension that requires evaluation.

These techniques seek reliability through different mechanisms. Self-consistency and debate broaden the set of candidate reasoning paths; Chain-of-Agents distributes evidence processing across long inputs; reflection and critique revisit a provisional answer to identify and repair errors (Dhuliawala et al., 2024; Du et al., 2024; Madaan et al., 2023; Shinn et al., 2023; X. Wang et al., 2023; Y. Zhang et al., 2024). They are not interchangeable: sampling more paths helps only when those paths provide meaningful diversity, sequential evidence transfer can lose information, and a critic may share the generator's blind spots. Combining mechanisms is possible, but each extra pass adds inference cost and requires an aggregation or stopping rule. The appropriate choice therefore

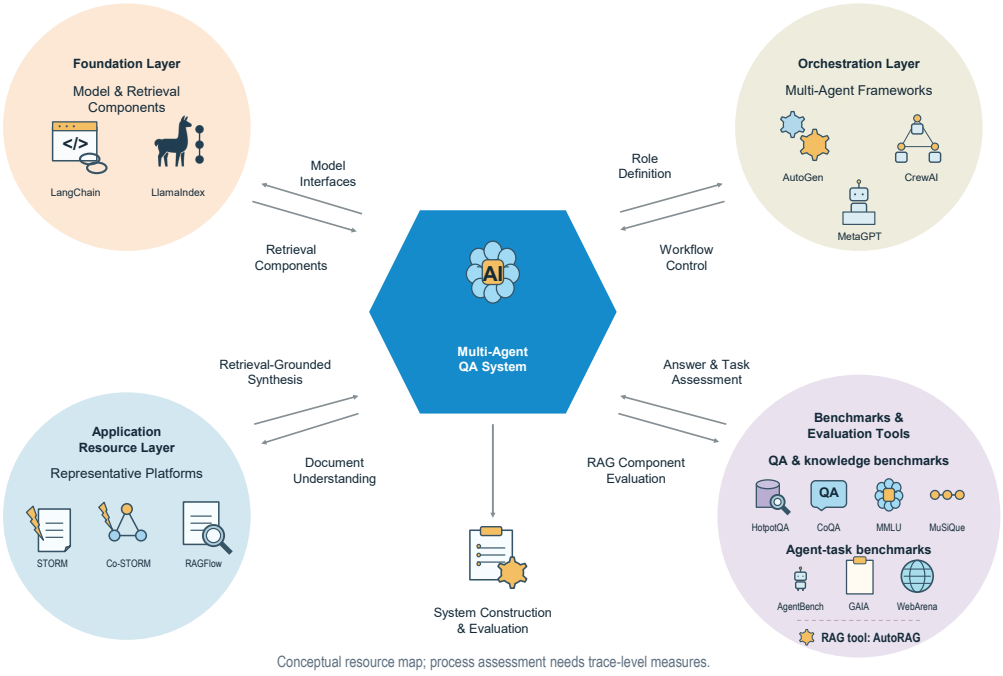


Figure 5: Four resource layers for constructing and evaluating multi-agent collaborative QA systems.

depends on the dominant failure mode: ambiguity among plausible answers, limited capacity to process long evidence, or the need to audit a provisional response. Reliability should be evaluated against these failure modes rather than inferred from the number of agents or reasoning rounds.

5. Development and evaluation resources for multi-agent collaborative QA

The preceding sections examined how multi-agent collaborative QA systems are organized, implemented, selected, and applied. This section reviews the resources needed to construct and assess such systems, following the four layers shown in Fig. 5: foundation components, multi-agent orchestration frameworks, application-oriented resources, and benchmarks and evaluation tools. The layers serve complementary functions: foundation components connect models and retrieval resources; orchestration frameworks define agent roles and workflow control; application resources instantiate task-oriented capabilities; and benchmarks and tools assess answer quality and collaboration. The figure is a conceptual map, not a requirement that every system use a single tool from each layer.

5.1. Foundation layer: model and retrieval components

The foundation layer supplies the interfaces and reusable components on which a QA workflow can be built. As illustrated in Fig. 5, development frameworks such as LangChain and LlamaIndex provide representative model-interface and retrieval-component resources. At this layer, the main concerns are connecting model services, preparing and retrieving evidence, and making external data accessible to the system. These components support retrieval-grounded answering, but do not by themselves assign responsibilities to multiple agents or determine how their work is coordinated.

Interoperability protocols address part of this integration problem. For example, the Model Context Protocol (MCP) standardizes connections between models or agents and external tools and data sources (Anthropic, 2024). Such interfaces can support several orchestration frameworks, but do not specify role allocation, task decomposition, or decision authority. These coordination functions belong to the orchestration layer discussed next.

5.2. Orchestration layer: multi-agent frameworks

The orchestration layer determines how agents are defined, assigned work, and coordinated during execution. Figure 5 illustrates this layer with AutoGen, CrewAI, and MetaGPT (Hong et al., 2024; Moura, 2023; Wu et al., 2024). Across the wider set of resources, some frameworks encode collaboration through predefined roles and procedures: for example, CAMEL uses role-conditioned dialogue, while ChatDev and MetaGPT organize work through staged, role-based processes (Hong et al., 2024; Li et al., 2023; Qian et al., 2024). Other frameworks expose more control flow to developers: AutoGen supports configurable conversations, LangGraph represents workflows as state graphs, and AgentVerse provides a phased collaboration process (W. Chen et al., 2024; Duan & Wang, 2024; Wu et al., 2024). GPTSwarm explores optimization of agent connections. DSPy, by contrast, focuses on metric-driven optimization of language-model

Table 4
Technical implementation comparison of representative multi-agent orchestration frameworks and related toolkits.

Framework	Control / organisation	Communication	Task decomposition	Memory	Typical QA application
CAMEL (Li et al., 2023)	Role-paired workflow	Role-play dialogue	Role-based task breakdown	Role-conditioned dialogue context	Exploratory dialogue; task simulation
ChatDev (Qian et al., 2024)	Hierarchical	Role-based ChatChain	Waterfall-style decomposition	Within-phase dialogue; cross-phase solutions	Software development; code generation
MetaGPT (Hong et al., 2024)	Hierarchical	Structured SOP messages	Role and responsibility mapping	Shared message pool and project documents	Software engineering
AutoGen (Wu et al., 2024)	Configurable	Configurable agent conversations	Configurable orchestration and speaker selection	Dialogue context; external tools	Conversational reasoning; code debugging
AgentVerse (W. Chen et al., 2024)	Adaptive role pipeline	Explicit messaging and debate	Expert recruitment; phase switching	Conversation and evaluation context	NLP evaluation; multi-step reasoning
LangGraph (The LangChain Team, 2024a, 2024b)	Configurable state graph	State-graph transitions	Developer-defined workflow nodes	Persistent state and checkpoint recovery	Long-running QA workflows
GPTSwarm (Zhuge et al., 2024)	Optimizable agent graph	Computational graph connections	Automatic graph-structure search	Intra-node state	MMLU QA; graph optimization
DSPy (Khattab et al., 2024)	Modular pipeline toolkit	Declarative module calls	Metric-driven prompt optimization	Module state and optimized demonstrations	QA and RAG pipeline optimization

pipelines and is therefore an adjacent toolkit, not a like-for-like multi-agent framework (Khattab et al., 2024; Zhuge et al., 2024).

Table 4 compares eight representative orchestration frameworks and related toolkits along five implementation dimensions: control / organisation, communication, task decomposition, memory or state handling, and typical QA-related use.

The comparison highlights three distinctions. First, frameworks vary in how much control flow they prescribe. Fixed-role or sequential designs make responsibilities and handoffs explicit, which can aid inspection in stable workflows; configurable and graph-based designs permit conditional routing and state transitions, but require developers to specify termination, recovery, and observability. These patterns are complementary, not a simple progression from less to more capable.

Second, communication, decomposition, and memory are implementation capabilities, not guarantees of effective collaboration. Structured messages, shared state, or persistent checkpoints can support coordination and recovery, but their usefulness depends on the application and evidence workflow. Foundation-layer interfaces and protocols such as MCP connect orchestration frameworks to tools and data sources, rather than replacing their workflow control logic (Anthropic, 2024).

Third, the resources differ in scope. DSPy optimizes language-model pipelines against task metrics and is included as a related pipeline toolkit, whereas the other entries provide multi-agent coordination or workflow capabilities. Accordingly, Table 4 compares implementation features, not controlled QA performance. Selection should begin with the required control pattern and operational constraints, then assess support for evidence handling, state recovery, observability, and task-level evaluation.

5.3. Application resource layer: representative platforms

The application resource layer contains systems that instantiate particular combinations of retrieval, multi-perspective reasoning, user interaction, and document processing. These systems complement the foundation and orchestration layers: unlike general-purpose frameworks, they package capabilities around an information-seeking or knowledge-service workflow. They are resources for constructing an application, not evidence that a particular architecture or platform will perform best. A reliable QA deployment still requires appropriate domain data, evidence policies, and evaluation protocols.

Representative resource configurations include STORM, Co-STORM, and RAGFlow. STORM and Co-STORM exemplify multi-perspective, retrieval-grounded information seeking and synthesis; RAGFlow is an open-source RAG engine oriented towards deep document understanding and knowledge-base workflows. They should therefore be read as representative resource configurations rather than as a

controlled ranking of systems. The choice among them depends on whether the intended QA task emphasizes open-ended research and synthesis, user-steered exploration, or document-rich enterprise knowledge access.

STORM (Synthesis of Topic Outlines through Retrieval and Multi-perspective Question Asking) structures long-form, source-grounded writing around perspective discovery, simulated question–answer conversations, and outline-guided synthesis (Shao et al., 2024). Its design is particularly relevant when a system must expand an initially broad topic into a structured research report. Co-STORM extends this setting by allowing users to observe and steer a discourse among language-model agents; a dynamic mind map records the information uncovered during the interaction (Y. Jiang et al., 2024). This makes it a useful reference point for human-in-the-loop (HITL) information exploration, in which the user can redirect the collaboration rather than only accept its final output.

In Co-STORM, a dynamic mind map records information uncovered during the discourse and makes the evolving interaction more visible to the user. This representation can support exploration, but does not by itself provide a complete audit trail. High-stakes deployments still require explicit provenance, approval, and responsibility mechanisms, as discussed in Section 6.

RAGFlow addresses a different but complementary engineering requirement. As an open-source RAG engine, it places deep document understanding at the center of knowledge ingestion and retrieval workflows (InfiniFlow, 2024). Its document-processing orientation is relevant when QA quality depends on extracting usable evidence from heterogeneous enterprise or technical documents. In this context, deep document understanding (DDU) denotes document parsing that aims to preserve structures such as layouts, tables, and other content required for subsequent retrieval and answer generation.

Table 5 compares the distinct configurations represented by these systems. The comparison is descriptive: it identifies the resource and interaction assumptions that a practitioner would need to inspect when translating an architecture choice into a concrete implementation.

5.4. Benchmarks and evaluation

5.4.1. Answer-focused and agent-oriented benchmarks

Benchmark resources should be matched to the system property being assessed. Conventional QA datasets are useful for testing retrieval, multi-hop reasoning, conversational context handling, and broad knowledge coverage. Agent-oriented environments additionally test planning, tool use, and long-horizon interaction. Neither class alone is sufficient for assessing a multi-agent QA system: a system can answer a static question correctly while using an inefficient collaboration process, or it can execute tools successfully without providing reliable evidence for an answer.

Representative conventional benchmarks cover distinct answer-level capabilities. HotpotQA provides Wikipedia-based multi-hop questions with sentence-level supporting facts, making it appropriate for evaluating evidence-grounded retrieval and reasoning (Z. Yang et al., 2018). CoQA evaluates context-dependent conversational QA through connected question–answer turns (Reddy et al., 2019). MMLU tests broad multi-task knowledge and reasoning across academic subjects (Hendrycks et al., 2021), whereas MuSiQue constructs multi-hop questions from single-hop components to probe connected reasoning (Trivedi et al., 2022). These resources assess answer-focused capabilities, but provide limited direct evidence about an agent team's planning or tool-use process.

Examples of agent-oriented benchmarks include AgentBench, GAIA, and WebArena. AgentBench evaluates LLM agents across eight environments involving reasoning and decision-making (Liu et al., 2024); GAIA combines reasoning with multi-modal information and tool-use requirements for general AI assistants (Mialon et al., 2024); and WebArena supplies reproducible web environments for long-horizon tasks across common online domains (S. Zhou et al., 2024). Together, these benchmarks enable a more complete resource map, although their task definitions, interfaces, and success metrics differ and should not be merged into a single aggregate score.

5.4.2. Automated evaluation and optimisation tools

Static benchmarks can be complemented by tools that automate the comparison of retrieval-augmented generation (RAG) configurations. For example, AutoRAG searches for RAG-module combinations that are suitable for a given dataset and reports the resulting evaluation outcomes (D. Kim et al., 2024). For multi-agent QA, such tooling can be used to evaluate retrieval and generation components inside a broader collaboration workflow. It should not, however, be taken as an evaluation of the collaboration policy itself: agent selection, communication, disagreement resolution, and human intervention require trace-level measures in addition to final-answer scores.

Evaluation implications. Answer-focused benchmarks assess retrieval, reasoning, and answer quality, whereas agent-oriented environments assess planning, tool use, and interaction over longer task horizons. Automated RAG optimisation can compare retrieval and generation configurations, but does not by itself assess the collaboration policy. A comprehensive evaluation should therefore combine answer-level scores with process measures such as agent selection, communication cost, evidence traceability, disagreement handling, and recovery from failure. The appropriate resources depend on the task, evidence source, risk level, and available budget; no single platform or benchmark captures all of these properties.

6. Challenges and Future Directions

Despite rapid progress, multi-agent collaborative QA still faces substantial challenges before it can be deployed reliably beyond controlled demonstrations. The challenges and research directions in this section are organized around the architecture taxonomy in Section 3 and the five technical dimensions in Section 4. They address architecture choice, the

Table 5
Representative platform configurations for multi-agent collaborative QA.

Capability dimension	STORM	Co-STORM	RAGFlow
Primary resource orientation	Retrieval-grounded long-form knowledge synthesis	User-participatory information exploration and report synthesis	Document-centred RAG and knowledge-base workflows
Collaboration or workflow mechanism	Multi-perspective questioning, simulated conversations, and outline-guided writing	Multi-agent discourse with a moderator and a dynamic mind map for interaction tracking	Document ingestion, retrieval, and configurable RAG components
Human-control point	Primarily topic and report specification before synthesis	Users may observe and steer the ongoing discourse	Workflow and knowledge-base configuration for deployment contexts
Best-matched QA setting	Broad research or explanatory questions requiring structured synthesis	Exploratory questions for which user intent may evolve during interaction	QA over heterogeneous, document-rich organizational knowledge
Selection boundary	Does not substitute for robust local-document parsing or high-stakes review controls	Human participation requires explicit intervention and accountability policies	Document understanding alone does not establish multi-agent reasoning quality

Table 6
Benchmark resources and their primary evaluation role in multi-agent collaborative QA.

Resource	Evaluation setting	Primary capability assessed	Interpretation for multi-agent QA
HotpotQA	Multi-hop QA with supporting facts	Cross-document retrieval, evidence use, and multi-hop reasoning	Tests whether collaboration improves evidence-grounded answer construction
CoQA	Conversational QA	Context retention, co-reference handling, and answer consistency across turns	Tests memory and dialogue coordination under connected QA turns
MMLU	Multi-subject multiple-choice tasks	Broad knowledge and subject-specific reasoning	Provides a knowledge-reasoning baseline, not a direct measure of collaboration quality
MuSiQue	Compositional multi-hop QA	Connected reasoning across linked subquestions	Tests decomposition and evidence chaining relevant to specialized agent roles
AgentBench	Interactive agent environments	Reasoning, decision-making, and environment interaction	Tests tool- and environment-facing agent behaviour beyond static answers
GAIA	General-assistant tasks	Reasoning, information access, and tool use	Tests integrated task completion requiring heterogeneous capabilities
WebArena	Reproducible web environments	Long-horizon web interaction and functional task completion	Tests planning and action execution in realistic online workflows

operation of coordination mechanisms, information management, reasoning and evaluation, human–agent collaboration, and adaptation across successive tasks.

6.1. Open challenges

The challenges are separated into architecture-level and mechanism-level issues. Architecture-level difficulty concerns choosing or adapting the distribution of decision authority; mechanism-level difficulty concerns how agents communicate, divide work, manage information, and reason within that architecture. Human–agent collaboration and continual learning cut across these levels but raise distinct oversight and temporal-adaptation questions.

6.1.1. Architecture selection under competing requirements

The taxonomy in Section 3 distinguishes flat, hierarchical, and hybrid architectures by their distribution of decision authority and control. In practice, a task may require open-ended exploration, reliable convergence, specialized knowledge, low latency, and risk control at the same time. EduPlanner (X. Zhang et al., 2025) combines open-ended instructional design with curriculum-scoring objectives, while Financial Reflection QA (Fatemi & Hu, 2024) combines a convergent numerical objective with critic feedback and answer revision. These cases illustrate why a single criterion or categorical label may not capture the control structure a task requires. Static architecture selection can also be inadequate when the workflow needs to change topology or authority across stages.

6.1.2. Communication and task coordination

Sections 4.1 and 4.2 show that communication, decomposition, assignment, and scheduling are distinct but interdependent. A decomposition may omit necessary subquestions or create unnecessary work; role assignment may not match agent competence; and a communication protocol may transmit messages without resolving dependencies or disagreement. In a fully connected broadcast topology, potential pairwise messages per round grow quadratically with team size, potentially increasing token and latency costs

without guaranteeing information gain (Qian et al., 2025). Static workflows are easier to inspect, but may not adapt to changing evidence or query difficulty. Dynamic recruitment and routing can respond to these changes, yet introduce decisions about escalation, termination, and recovery.

6.1.3. Knowledge and memory lifecycle

Sections 4.3 and 4.4 distinguish shared knowledge from agent or workflow memory, but systems may combine them in the same stores and retrieval pipelines. External documents, retrieved evidence, generated claims, dialogue history, and learned experience have different provenance, validity, and retention requirements. Without these distinctions, stale or unsupported claims may be reused as facts, private context may be shared too broadly, and updates may overwrite useful state. Conversely, isolating every agent's information can cause repeated retrieval, inconsistent evidence, and loss of context across turns. The unresolved questions concern how information moves from retrieved evidence to shared knowledge, how long it remains in memory, who may access or revise it, and how corrections are propagated.

6.1.4. Reasoning reliability and process evaluation

Section 4.5 covers diverse reasoning paths, sequential evidence processing, reflection, and mutual critique. These mechanisms address different failure modes. Conventional benchmarks and agent environments measure answer correctness, supporting-fact accuracy, or task completion (Hendrycks et al., 2021; Liu et al., 2024; Mialon et al., 2024; Z. Yang et al., 2018); these outcome measures alone do not establish whether debate added independent evidence, whether a long-context chain lost a decisive fact, or whether a critic detected and corrected an error. Additional reasoning rounds can also repeat the same assumptions or share correlated blind spots. Trace analyses such as MAST identify design, alignment, and verification failures (Cemri et al., 2025), suggesting that process-level measures of contribution, complementarity, and error correction could complement outcome measures.

As a result, a system may improve a benchmark score while using substantially more agents, tokens, or tool calls, with little evidence about which mechanism produced the gain. It remains difficult to compare reasoning strategies under matched models, evidence, and resource budgets or to determine when debate, sequential processing, or critique is warranted.

6.1.5. Human–agent collaboration and accountability

The human-in-the-loop pattern in Section 3.1 and the evidence and reasoning mechanisms in Section 4 make review possible, but a breakpoint alone does not provide accountable oversight. Reviewers need to inspect relevant sources, agent contributions, uncertainty, and state changes, and they need authority appropriate to the decision. Medical and legal QA provide application contexts for these proposed oversight requirements (Cui et al., 2023; Tang et al., 2024). Frameworks support configurable interaction and workflow control (Duan & Wang, 2024; The LangChain Team, 2024b). Their use still requires designers to specify when intervention is useful, what can be revised, and how the intervention affects subsequent reasoning. Oversight that is too late cannot correct upstream retrieval or decomposition errors, while review at every stage may impose excessive latency and workload.

6.1.6. Continual learning and team adaptation

Persistent systems may process successive cases with the same knowledge and collaboration policy even when verified outcomes and human corrections could inform later decisions. Updating individual memory without provenance can propagate errors; changing routing, voting, or role authority without longitudinal validation can reinforce collective bias, cause forgetting, or destabilize team behaviour. This challenge is related to memory management in Section 4.4, but differs from within-task retention: it concerns validated adaptation across cases and over time.

6.2. Future research directions

The directions below follow the same order as the challenges and respond to both architecture-level and mechanism-level limitations.

6.2.1. Architecture-aware selection and adaptation

The architecture-selection approach in Section 3 should be tested beyond a one-time choice among flat, hierarchical, and hybrid labels. Future work could operationalise task complexity, process dynamism, cost, and risk as explicit inputs to architecture selection, while allowing control authority or topology to change when task stages require different forms of collaboration. The comparison should retain a single-agent baseline and report where a fixed architecture is sufficient, where a hybrid design is warranted, and where additional agents do not justify their cost.

Validation should use larger and more diverse system samples, independent coders, and prospective task-level comparisons under matched models and budgets. This would test the framework's boundary conditions and distinguish the descriptive taxonomy from an empirically supported selection policy.

Declarative system composition could make this selection operational: users would specify the task, domain sources, required roles, permissions, risk, and resource budget, and a platform would assemble an initial architecture and workflow for review. Existing frameworks provide reusable components for such configurations (LangGenius, Inc., 2024; Moura, 2023; The LangChain Team, 2024a; Wu et al., 2024), but the resulting designs need validation on historical or sandbox tasks before deployment.

6.2.2. Joint communication and task orchestration

Future orchestration should jointly select decomposition depth, agent roles, communication pattern, and stopping conditions, rather than treating these as independent configuration choices. A controller could estimate task complexity, evidence sufficiency, disagreement, and resource use, then recruit a specialist, change the interaction pattern, or stop when further work is unlikely to improve the answer. MDAgents provides an initial example of adapting collaboration

to problem difficulty (Y. Kim et al., 2024); scaling results likewise indicate that the value of more agents depends on task and coordination design (Qian et al., 2025).

Evaluation should compare static, rule-based, and learned coordination under matched models and resource budgets. In addition to answer quality, it should measure decomposition coverage, assignment fit, message volume, latency, recruitment overhead, and recovery from failed subtasks. This would connect the selection criteria in Section 3 to the concrete communication and scheduling mechanisms in Section 4.

6.2.3. Provenance-aware knowledge and memory management

Systems should represent retrieved evidence, shared knowledge, dialogue state, and learned experience as distinct information objects with explicit provenance, timestamps, confidence, and validation status. Policies should specify when an item remains local to an agent, when it can enter shared team knowledge, how long it is retained in memory, and how it can be corrected or withdrawn. Retrieval and memory updates should be evaluated for factual support, freshness, privacy, and their effect on later answers, not only for retrieval recall.

These policies govern information handling within a task and across system components. They do not, by themselves, specify how the system should learn from outcomes across cases; that longitudinal problem is addressed separately below.

6.2.4. Mechanism-level reasoning and process evaluation

Evaluation should measure how each reasoning mechanism contributes to the answer, not only whether the final answer is correct. Structured traces can record claims, supporting sources, confidence, responses, revisions, and whether each contribution is retained in the final answer. Measures should include marginal quality gain by round, evidence coverage, error-correction rate, useful disagreement, redundant messages, and resource cost. Failure taxonomies such as MAST provide a starting point (Cemri et al., 2025), but positive measures of complementarity and successful convergence are also needed.

Comparisons should hold the base model, evidence, and inference budget constant while testing self-consistency or debate, sequential evidence processing, reflection, and their combinations. Human review of selected traces can determine whether apparent gains reflect new evidence or rephrasing. The resulting metrics should be tested across multi-hop, long-document, open-domain, and specialized QA to establish which mechanism helps under which failure conditions.

6.2.5. Risk-adaptive human–agent collaboration

Human review should be triggered according to task risk, evidence uncertainty, and inter-agent disagreement, and should be placed where the relevant state is still inspectable and revisable. Review may be useful before decomposition, after evidence retrieval, following unresolved critique, before a high-impact tool action, or before release of the answer. Each intervention should record the evidence inspected, the state changed, the reviewer's authority and rationale, and the effect on subsequent communication, memory, and reasoning.

Evaluation should compare fixed breakpoints, uncertainty-triggered review, disagreement-triggered review, and continuous supervision across risk levels. Outcomes should include error interception, decision time, workload, answer quality, and completeness of provenance and responsibility records. Such studies would connect the human–agent collaboration pattern to the communication, knowledge, and reasoning traces reviewers need in order to act effectively.

6.2.6. Continual learning of agents and team strategies

Persistent QA systems could use verified cases, retrieved updates, and human corrections to refine individual domain memories, record recurring errors, and calibrate expertise boundaries. Work on agent memory and reflective feedback offers components for such updating (Shinn et al., 2023; Z. Zhang et al., 2025); each update should retain its source, timestamp, confidence, and validation status.

At the group level, systems could learn which agents are reliable for particular question types, which combinations contribute complementary evidence, and which interaction patterns repeat errors. These histories could inform routing, speaking order, voting weights, and stopping rules, extending individual memory towards learned team coordination (W. Chen et al., 2024; Park et al., 2023). Longitudinal studies should compare individual updating, team-policy updating, and their combination on sequential tasks with changing knowledge. They should measure adaptation speed, retention, long-term accuracy, and cost, while testing for error propagation, collective bias, catastrophic forgetting, privacy leakage, and unstable shifts in agent authority.

6.3. Limitations of this survey

This survey has four principal limitations. First, the literature search extends to mid-2026, and the speed of development means that later or insufficiently indexed systems cannot be represented. Second, architecture classification depends on the level of implementation detail reported by the original authors. Systems that dynamically switch topology or combine several coordination mechanisms may therefore remain ambiguous at category boundaries. Third, the ten-case comparison in Section 3 is a limited verification exercise rather than independent confirmation of general validity; the three-tier ratings describe organisational compatibility, and this sample contains no source-coded hybrid example. Broader testing across tasks, models, and resource budgets is required. Fourth, the architecture-selection guide remains qualitative. Its decision factors have not yet been operationalised as a weighted model learned or calibrated on a large labeled system corpus. These limitations bound the claims that can be made for the proposed taxonomy and selection guide, while motivating their future empirical refinement.

7. Conclusion

This survey examined multi-agent collaborative QA through three connected questions: why collaboration is needed, how it is organised, and when it is justified. The architecture taxonomy distinguishes flat, hierarchical, and hybrid control patterns by the distribution of decision authority. The accompanying selection approach treats the single-agent workflow as a valid baseline and uses task complexity, process structure, cost, and risk as explicit criteria. Its comparison with ten published systems shows that the criteria can make design assumptions and boundary cases visible, but cannot replace task-level evaluation or guarantee a universally appropriate topology.

The survey also reviewed the technical mechanisms that support collaboration and organized practical resources into foundation components, orchestration frameworks, application resources, and benchmarks and evaluation tools. This separation clarifies that a framework or platform provides implementation capabilities, whereas answer quality and collaboration quality require independent evaluation. The main unresolved problems include selecting and adapting architectures under competing requirements, coordinating agents within resource constraints, governing shared knowledge and memory, evaluating reasoning processes, designing effective human–agent collaboration, and learning reliably across successive tasks. Progress on these problems is necessary for multi-agent QA to move from promising demonstrations toward dependable deployment in domain-specific and high-stakes settings.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The reference-verification table and search log underlying the evidence map are provided with the submission as `literature_audit.csv` and `search_log.csv`. The current field-level and claim-level audit tables are `reference_field_audit.csv` and `citation_claim_audit.csv`; they record unresolved optional publication fields, documented version selections, and evidence boundaries.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used large language model (LLM) tools to assist with grammar checking, language refinement, and reference formatting, and with plotting code and layout for the data-derived reference-corpus diagram (Fig. 1), and used Nano Banana to generate the initial draft of an explanatory architecture schematic during drafting. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

Anthropic. (2024, November 25). *Introducing the Model Context Protocol*. Retrieved October 11, 2026, from https://www.anthropic.com/news/model-context-protocol

Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, & Y. Sun (Eds.), *International Conference on Learning Representations* (pp. 9112–9141). International Conference on Learning Representations. https://proceedings.iclr.cc/paper_files/paper/2024/hash/25f7be9694d7b32d5cc670927b8091e1-Abstract-Conference.html

Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Podstawski, M., Gianinazzi, L., Gajda, J., Lehmann, T., Niewiadomski, H., Nyczyk, P., & Hoefler, T. (2024). Graph of thoughts: Solving elaborate problems with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, *38*(16), 17682–17690. https://doi.org/10.1609/aaai.v38i16.29720

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., … Amodei, D. (2020). Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, & H. Lin (Eds.), *Advances in neural information processing systems* (pp. 1877–1901, Vol. 33). Curran Associates, Inc. https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html

Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M. A., Gonzalez, J. E., & Stoica, I. (2025). Why do multi-agent LLM systems fail? In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, & N. Chen (Eds.), *Advances in neural information processing systems* (pp. 135676–135729, Vol. 38). Curran Associates, Inc. https://doi.org/10.52202/085713-4082

Chen, J., Saha, S., & Bansal, M. (2024). ReConcile: Round-table conference improves reasoning via consensus among diverse LLMs. In L.-W. Ku, A. Martins, & V. Srikumar (Eds.), *Proceedings of the 62nd annual meeting of the Association for Computational Linguistics (volume 1: Long papers)* (pp. 7066–7085). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.acl-long.381

Chen, W., Su, Y., Zuo, J., Yang, C., Yuan, C., Chan, C.-M., Yu, H., Lu, Y., Hung, Y.-H., Qian, C., Qin, Y., Cong, X., Xie, R., Liu, Z., Sun, M., & Zhou, J. (2024). AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, & Y. Sun (Eds.), *International Conference on Learning Representations* (pp. 20094–20136). International Conference on Learning Representations. https://proceedings.iclr.cc/paper_files/paper/2024/hash/578e65cdee35d00c708d4c64bce32971-Abstract-Conference.html

Chen, Z., Zhang, Q., Xiang, Z., Wei, Z., Gao, L., Huang, X., Zhang, Z., & Su, J. (2026). LegalGraphRAG: Multi-agent graph retrieval-augmented generation for reliable legal reasoning. In M. Liakata, V. P. Moreira, J. Zhang, & D. Jurgens (Eds.), *Proceedings of the 64th annual meeting of the Association for Computational Linguistics (volume 1: Long papers)* (pp. 37455–37484). Association for Computational Linguistics. https://doi.org/10.18653/v1/2026.acl-long.1738

Choi, E., He, H., Iyyer, M., Yatskar, M., Yih, W.-t., Choi, Y., Liang, P., & Zettlemoyer, L. (2018). QuAC: Question answering in context. In E. Riloff, D. Chiang, J. Hockenmaier, & J. Tsujii (Eds.),

Proceedings of the 2018 conference on empirical methods in natural language processing (pp. 2174–2184). Association for Computational Linguistics. https://doi.org/10.18653/v1/D18-1241

Cui, J., Li, Z., Yan, Y., Chen, B., & Yuan, L. (2023). *ChatLaw: Open-source legal large language model with integrated external knowledge bases*. arXiv. https://doi.org/10.48550/arXiv.2306.16092

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In J. Burstein, C. Doran, & T. Solorio (Eds.), *Proceedings of the 2019 conference of the north American chapter of the Association for Computational Linguistics: Human language technologies, volume 1 (long and short papers)* (pp. 4171–4186). Association for Computational Linguistics. https://doi.org/10.18653/v1/N19-1423

Dhuliawala, S., Komeili, M., Xu, J., Raileanu, R., Li, X., Celikyilmaz, A., & Weston, J. (2024). Chain-of-verification reduces hallucination in large language models. In L.-W. Ku, A. Martins, & V. Srikumar (Eds.), *Findings of the Association for Computational Linguistics: ACL 2024* (pp. 3563–3578). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.findings-acl.212

Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2024). Improving factuality and reasoning in language models through multiagent debate. In R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, & F. Berkenkamp (Eds.), *Proceedings of the 41st international conference on machine learning* (pp. 11733–11763, Vol. 235). PMLR. https://proceedings.mlr.press/v235/du24e.html

Duan, Z., & Wang, J. (2024). *Exploration of LLM multi-agent application implementation based on LangGraph+CrewAI*. arXiv. https://doi.org/10.48550/arXiv.2411.18241

Fatemi, S., & Hu, Y. (2024). Enhancing financial question answering with a multi-agent reflection framework. In *Proceedings of the 5th ACM international conference on AI in finance* (pp. 530–537). Association for Computing Machinery. https://doi.org/10.1145/3677052.3698686

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2023). *Retrieval-augmented generation for large language models: A survey*. arXiv. https://doi.org/10.48550/arXiv.2312.10997

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., & Zhang, X. (2024). Large language model based multi-agents: A survey of progress and challenges. In K. Larson (Ed.), *Proceedings of the thirty-third international joint conference on artificial intelligence, IJCAI-24* (pp. 8048–8057). International Joint Conferences on Artificial Intelligence Organization. https://doi.org/10.24963/ijcai.2024/890

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., & Steinhardt, J. (2021). Measuring massive multitask language understanding. In *International Conference on Learning Representations*. International Conference on Learning Representations. https://openreview.net/forum?id=d7KBjmI3GmQ

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., & Schmidhuber, J. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, & Y. Sun (Eds.), *International Conference on Learning Representations* (pp. 23247–23275). International Conference on Learning Representations. https://proceedings.iclr.cc/paper_files/paper/2024/hash/6507b115562bb0a305f1958ccc87355a-Abstract-Conference.html

Huang, D., Bu, Q., Zhang, J. M., Luck, M., & Cui, H. (2023). *AgentCoder: Multi-agent-based code generation with iterative testing and optimisation*. arXiv. https://doi.org/10.48550/arXiv.2312.13010

InfiniFlow. (2024). *RAGFlow: An open-source RAG engine based on deep document understanding* (Version 0.6.0) [Computer software]. GitHub. https://github.com/infiniflow/ragflow/tree/v0.6.0

Jiang, Y., Zhao, S., Li, J., Wang, H., Zhang, L., Liu, Y., & Qin, B. (2026). Collaborative chain-of-agents for parametric-retrieved knowledge synergy. In M. Liakata, V. P. Moreira, J. Zhang, & D. Jurgens (Eds.), *Proceedings of the 64th annual meeting of the Association for Computational Linguistics (volume 1: Long papers)* (pp. 3663–3680). Association for Computational Linguistics. https://doi.org/10.18653/v1/2026.acl-long.167

Jiang, Y., Shao, Y., Ma, D., Semnani, S., & Lam, M. (2024). Into the unknown unknowns: Engaged human learning through participation in language model agent conversations. In Y. Al-Onaizan, M. Bansal, & Y.-N. Chen (Eds.), *Proceedings of the 2024 conference on empirical methods in natural language processing* (pp. 9917–9955). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.emnlp-main.554

Jiang, Z., Xu, F. F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., & Neubig, G. (2023). Active retrieval augmented generation. In H. Bouamor, J. Pino, & K. Bali (Eds.), *Proceedings of the 2023 conference on empirical methods in natural language processing* (pp. 7969–7992). Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.emnlp-main.495

Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. In B. Webber, T. Cohn, Y. He, & Y. Liu (Eds.), *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)* (pp. 6769–6781). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-main.550

Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M., & Potts, C. (2024). DSPy: Compiling declarative language model calls into self-improving pipelines. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, & Y. Sun (Eds.), *International Conference on Learning Representations* (pp. 54928–54958). International Conference on Learning Representations. https://proceedings.iclr.cc/paper_files/paper/2024/hash/f1cf02ce09757f57c3b93c0db83181e0-Abstract-Conference.html

Khot, T., Trivedi, H., Finlayson, M., Fu, Y., Richardson, K., Clark, P., & Sabharwal, A. (2023). Decomposed prompting: A modular approach for solving complex tasks. In *The eleventh International Conference on Learning Representations*. International Conference on Learning Representations. https://openreview.net/forum?id=_nGgzQjzaRy

Kim, D., Kim, B., Han, D., & Eibich, M. (2024). *AutoRAG: Automated framework for optimization of retrieval augmented generation pipeline*. arXiv. https://doi.org/10.48550/arXiv.2410.20878

Kim, Y., Park, C., Jeong, H., Chan, Y. S., Xu, X., McDuff, D., Lee, H., Ghassemi, M., Breazeal, C., & Park, H. W. (2024). MDAgents: An adaptive collaboration of LLMs for medical decision-making. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, & C. Zhang (Eds.), *Advances in neural information processing systems* (pp. 79410–79452, Vol. 37). Curran Associates, Inc. https://doi.org/10.52202/079017-2522

Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., Epstein, D., Polosukhin, I., Devlin, J., Lee, K., Toutanova, K., Jones, L., Kelcey, M., Chang, M.-W., Dai, A. M., Uszkoreit, J., Le, Q., & Petrov, S. (2019). Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, *7*, 453–466. https://doi.org/10.1162/tacl_a_00276

LangGenius, Inc. (2024). *Dify: Open-source LLM app development platform* (Version 0.6.0) [Computer software]. GitHub. https://github.com/langgenius/dify/tree/0.6.0

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, & H. Lin (Eds.), *Advances in neural information*

processing systems (pp. 9459–9474, Vol. 33). Curran Associates, Inc. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

Li, G., Hammoud, H., Itani, H., Khizbullin, D., & Ghanem, B. (2023). Camel: Communicative agents for "mind" exploration of large language model society. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, & S. Levine (Eds.), *Advances in neural information processing systems* (pp. 51991–52008, Vol. 36). Curran Associates, Inc. https://doi.org/10.52202/075280-2264

Liang, T., He, Z., Jiao, W., Wang, X., Wang, Y., Wang, R., Yang, Y., Shi, S., & Tu, Z. (2024). Encouraging divergent thinking in large language models through multi-agent debate. In Y. Al-Onaizan, M. Bansal, & Y.-N. Chen (Eds.), *Proceedings of the 2024 conference on empirical methods in natural language processing* (pp. 17889–17904). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.emnlp-main.992

Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su, Y., Sun, H., … Tang, J. (2024). AgentBench: Evaluating LLMs as agents. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, & Y. Sun (Eds.), *International Conference on Learning Representations* (pp. 52989–53046). International Conference on Learning Representations. https://proceedings.iclr.cc/paper_files/paper/2024/hash/e9df36b21ff4ee211a8b71ee8b7e9f57-Abstract-Conference.html

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., & Clark, P. (2023). Self-refine: Iterative refinement with self-feedback. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, & S. Levine (Eds.), *Advances in neural information processing systems* (pp. 46534–46594, Vol. 36). Curran Associates, Inc. https://doi.org/10.52202/075280-2019

Mialon, G., Fourrier, C., Swift, C., Wolf, T., LeCun, Y., & Scialom, T. (2024). GAIA: A benchmark for general AI assistants. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, & Y. Sun (Eds.), *International Conference on Learning Representations* (pp. 9025–9049). International Conference on Learning Representations. https://proceedings.iclr.cc/paper_files/paper/2024/file/25ae35b5b1738d80f1f03a8713e405ec-Paper-Conference.pdf

Minsky, M. (1988). *Society of mind*. Simon and Schuster. https://www.simonandschuster.com/books/Society-Of-Mind/Marvin-Minsky/9780671657130

Moura, J. (2023). *CrewAI: Framework for orchestrating role-playing, autonomous AI agents* (Version 0.1.0) [Computer software]. PyPI. https://pypi.org/project/crewai/0.1.0/

OpenAI. (2023). *GPT-4 technical report*. arXiv. https://doi.org/10.48550/arXiv.2303.08774

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., & Lowe, R. (2022). Training language models to follow instructions with human feedback. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, & A. Oh (Eds.), *Advances in neural information processing systems* (pp. 27730–27744, Vol. 35). Curran Associates, Inc. https://doi.org/10.52202/068431-2011

Packer, C., Fang, V., Patil, S. G., Lin, K., Wooders, S., & Gonzalez, J. E. (2023). *MemGPT: Towards LLMs as operating systems*. arXiv. https://doi.org/10.48550/arXiv.2310.08560

Park, J. S., O'Brien, J., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. In S. Follmer, J. Han, J. Steimle, & N. H. Riche (Eds.), *Proceedings of the 36th annual ACM symposium on user interface software and technology* (Article 2). Association for Computing Machinery. https://doi.org/10.1145/3586183.3606763

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, *24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., & Sun, M. (2024). ChatDev: Communicative agents for software development. In L.-W. Ku, A. Martins, & V. Srikumar (Eds.), *Proceedings of the 62nd annual meeting of the Association for Computational Linguistics (volume 1: Long papers)* (pp. 15174–15186). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.acl-long.810

Qian, C., Xie, Z., Wang, Y., Liu, W., Zhu, K., Xia, H., Dang, Y., Du, Z., Chen, W., Yang, C., Liu, Z., & Sun, M. (2025). Scaling large language model-based multi-agent collaboration. In Y. Yue, A. Garg, N. Peng, F. Sha, & R. Yu (Eds.), *International Conference on Learning Representations* (pp. 41488–41505). International Conference on Learning Representations. https://proceedings.iclr.cc/paper_files/paper/2025/hash/66a026c0d17040889b50f0dfa650e5e0-Abstract-Conference.html

Reddy, S., Chen, D., & Manning, C. D. (2019). CoQA: A conversational question answering challenge. *Transactions of the Association for Computational Linguistics*, *7*, 249–266. https://doi.org/10.1162/tacl_a_00266

Shao, Y., Jiang, Y., Kanell, T., Xu, P., Khattab, O., & Lam, M. (2024). Assisting in writing Wikipedia-like articles from scratch with large language models. In K. Duh, H. Gomez, & S. Bethard (Eds.), *Proceedings of the 2024 conference of the North American Chapter of the Association for Computational Linguistics: Human language technologies (volume 1: Long papers)* (pp. 6252–6278). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.naacl-long.347

Shen, Y., Song, K., Tan, X., Li, D., Lu, W., & Zhuang, Y. (2023). HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, & S. Levine (Eds.), *Advances in neural information processing systems* (pp. 38154–38180, Vol. 36). Curran Associates, Inc. https://doi.org/10.52202/075280-1657

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, & S. Levine (Eds.), *Advances in neural information processing systems* (pp. 8634–8652, Vol. 36). Curran Associates, Inc. https://doi.org/10.52202/075280-0377

Surapaneni, R., Jha, M., Vakoc, M., & Segal, T. (2025, April 9). *Announcing the Agent2Agent protocol (A2A)*. Google Developers Blog. Retrieved October 11, 2026, from https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/

Tang, X., Zou, A., Zhang, Z., Li, Z., Zhao, Y., Zhang, X., Cohan, A., & Gerstein, M. (2024). MedAgents: Large language models as collaborators for zero-shot medical reasoning. In L.-W. Ku, A. Martins, & V. Srikumar (Eds.), *Findings of the Association for Computational Linguistics: ACL 2024* (pp. 599–621). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.findings-acl.33

Team GLM. (2024). *ChatGLM: A family of large language models from GLM-130B to GLM-4 all tools*. arXiv. https://doi.org/10.48550/arXiv.2406.12793

The LangChain Team. (2024a, January 23). *LangGraph: Multi-agent workflows*. LangChain. Retrieved October 11, 2026, from https://www.langchain.com/blog/langgraph-multi-agent-workflows

The LangChain Team. (2024b, December 14). *Making it easier to build human-in-the-loop agents with interrupt*. LangChain. Retrieved October 11, 2026, from https://www.langchain.com/blog/

making-it-easier-to-build-human-in-the-loop-agents-with-interrupt

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., & Lample, G. (2023). *LLaMA: Open and efficient foundation language models*. arXiv. https://doi.org/10.48550/arXiv.2302.13971

Tran, K.-T., Dao, D., Nguyen, M.-D., Pham, Q.-V., O'Sullivan, B., & Nguyen, H. D. (2025). *Multi-agent collaboration mechanisms: A survey of LLMs*. arXiv. https://doi.org/10.48550/arXiv.2501.06322

Trivedi, H., Balasubramanian, N., Khot, T., & Sabharwal, A. (2022). MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, *10*, 539–554. https://doi.org/10.1162/tacl_a_00475

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In I. Guyon, U. von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), *Advances in neural information processing systems* (Vol. 30). Curran Associates, Inc. https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, *18*(6), Article 186345. https://doi.org/10.1007/s11704-024-40231-1

Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., & Lim, E.-P. (2023). Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In A. Rogers, J. Boyd-Graber, & N. Okazaki (Eds.), *Proceedings of the 61st annual meeting of the Association for Computational Linguistics (volume 1: Long papers)* (pp. 2609–2634). Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.acl-long.147

Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., & Zhou, D. (2023). Self-consistency improves chain of thought reasoning in language models. In *The eleventh International Conference on Learning Representations*. International Conference on Learning Representations. https://openreview.net/forum?id=1PL1NIMMrw

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, & A. Oh (Eds.), *Advances in neural information processing systems* (pp. 24824–24837, Vol. 35). Curran Associates, Inc. https://doi.org/10.52202/068431-1800

Wooldridge, M., & Jennings, N. R. (1995). Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, *10*(2), 115–152. https://doi.org/10.1017/S0269888900008122

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2024). AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In *First conference on language modeling*. https://openreview.net/forum?id=BAakY1hNKS

Xiong, K., Ding, X., Cao, Y., Liu, T., & Qin, B. (2023). Examining inter-consistency of large language models collaboration: An in-depth analysis via debate. In H. Bouamor, J. Pino, & K. Bali (Eds.), *Findings of the Association for Computational Linguistics: EMNLP 2023* (pp. 7572–7590). Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.findings-emnlp.508

Yang, H., Chen, H., Guo, H., Chen, Y., Lin, C.-S., Hu, S., Hu, J., Wu, X., & Wang, X. (2025). LLM-MedQA: Enhancing medical question answering through case studies in large language models. In *2025 international joint conference on neural networks (IJCNN)* (pp. 1–8). IEEE. https://doi.org/10.1109/IJCNN64981.2025.11228647

Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W., Salakhutdinov, R., & Manning, C. D. (2018). HotpotQA: A dataset for diverse, explainable multi-hop question answering. In E. Riloff, D. Chiang, J. Hockenmaier, & J. Tsujii (Eds.), *Proceedings of the 2018 conference on empirical methods in natural language processing* (pp. 2369–2380). Association for Computational Linguistics. https://doi.org/10.18653/v1/D18-1259

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, & S. Levine (Eds.), *Advances in neural information processing systems* (pp. 11809–11822, Vol. 36). Curran Associates, Inc. https://doi.org/10.52202/075280-0517

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *The eleventh International Conference on Learning Representations*. International Conference on Learning Representations. https://openreview.net/forum?id=WE_vluYUL-X

Zhang, X., Zhang, C., Sun, J., Xiao, J., Yang, Y., & Luo, Y. (2025). EduPlanner: LLM-based multiagent systems for customized and intelligent instructional design. *IEEE Transactions on Learning Technologies*, *18*, 416–427. https://doi.org/10.1109/TLT.2025.3561332

Zhang, Y., Sun, R., Chen, Y., Pfister, T., Zhang, R., & Arik, S. O. (2024). Chain of agents: Large language models collaborating on long-context tasks. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, & C. Zhang (Eds.), *Advances in neural information processing systems* (pp. 132208–132237, Vol. 37). Curran Associates, Inc. https://doi.org/10.52202/079017-4202

Zhang, Z., Dai, Q., Bo, X., Ma, C., Li, R., Chen, X., Zhu, J., Dong, Z., & Wen, J.-R. (2025). A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, *43*(6), Article 155. https://doi.org/10.1145/3748302

Zhao, J., Zu, C., Hao, X., Lu, Y., He, W., Ding, Y., Gui, T., Zhang, Q., & Huang, X.-J. (2024). LONGAGENT: Achieving question answering for 128k-token-long documents through multi-agent collaboration. In Y. Al-Onaizan, M. Bansal, & Y.-N. Chen (Eds.), *Proceedings of the 2024 conference on empirical methods in natural language processing* (pp. 16310–16324). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.emnlp-main.912

Zhou, D., Schärli, N., Hou, L., Wei, J., Scales, N., Wang, X., Schuurmans, D., Cui, C., Bousquet, O., Le, Q. V., & Chi, E. H. (2023). Least-to-most prompting enables complex reasoning in large language models. In *The eleventh International Conference on Learning Representations*. International Conference on Learning Representations. https://openreview.net/forum?id=WZH7099tgfM

Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., & Neubig, G. (2024). WebArena: A realistic web environment for building autonomous agents. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, & Y. Sun (Eds.), *International Conference on Learning Representations* (pp. 15585–15606). International Conference on Learning Representations. https://proceedings.iclr.cc/paper_files/paper/2024/hash/4410c0711e9154a7a2d26f9b3816d1ef-Abstract-Conference.html

Zhuge, M., Wang, W., Kirsch, L., Faccio, F., Khizbullin, D., & Schmidhuber, J. (2024). GPTSwarm: Language agents as optimizable graphs. In R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, & F. Berkenkamp (Eds.), *Proceedings of the 41st international conference on machine learning* (pp. 62743–62767, Vol. 235). PMLR. https://proceedings.mlr.press/v235/zhuge24a.html

Zong, C., Yan, Y., Lu, W., Shao, J., Huang, Y., Chang, H., & Zhuang, Y. (2024). Triad: A framework leveraging a multi-role LLM-based agent to solve knowledge base question answering. In Y. Al-Onaizan, M. Bansal, & Y.-N. Chen (Eds.), *Proceedings of the*

2024 conference on empirical methods in natural language processing (pp. 1698–1710). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.emnlp-main.101